



Embodied AI Practicals - Intro to Pytorch, 3D Simulator and ROS

Course AIAA 4220

Week5- Lecture 9 & 10

Changhao Chen

Assistant Professor

HKUST (GZ)

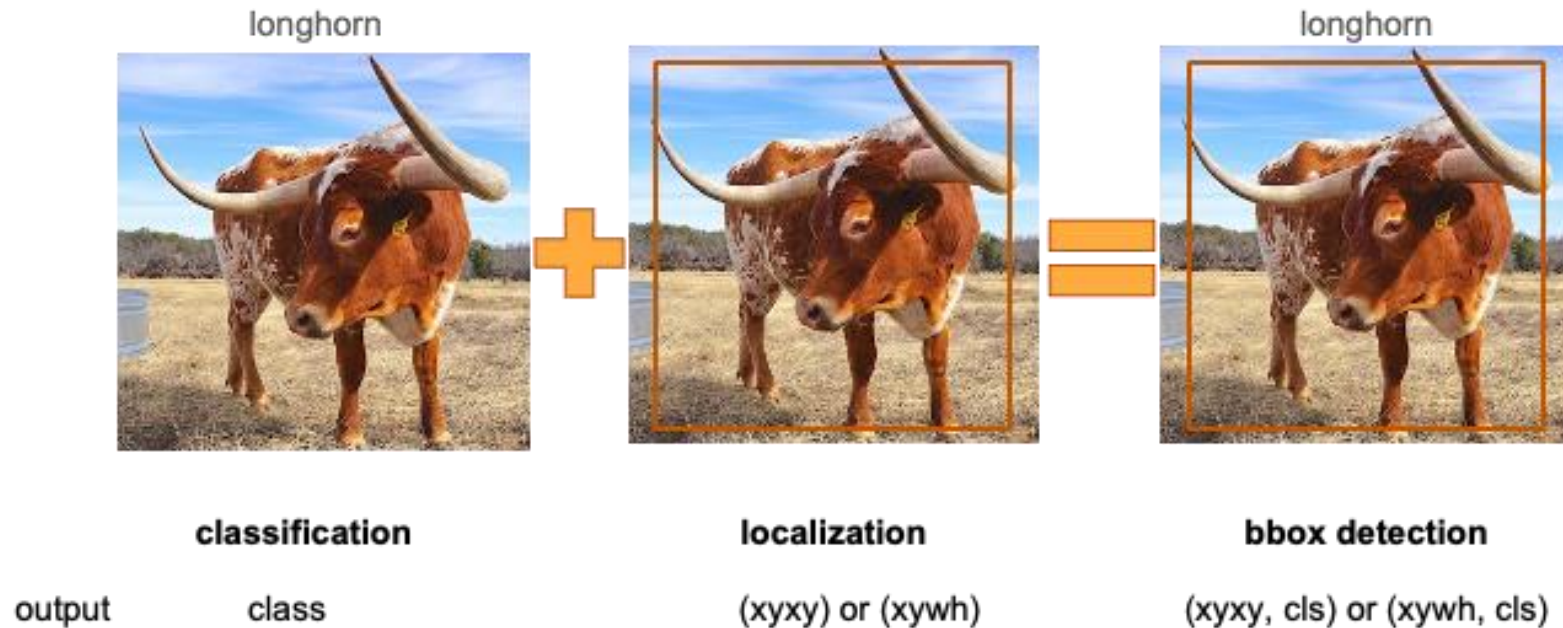
Course Overview

1. Recap: Object Detection and Tracking
2. Small Test
3. Pytorch Basics
4. Object Detection Training and Testing
5. Project 2 Release
6. 3D Simulator
7. Robot Operating System (ROS)

Recap: Object Detection

Task: Classification + Localization

Formal Definition: "What is it and where is it?"



- **Localization as Regression:** Predicting the bounding box coordinates (e.g., x , y) is a regression task.
- **Pixel Coordinate System:** The top-left corner of an image is defined as the origin (0, 0).
- **Bounding Box Representations:**
 - **Corners:** (x_1, y_1) of the top-left and (x_2, y_2) of the bottom-right.
 - **Center & Dimensions:** (x, y) of the center point, plus width (w) and height (h).

Recap: Object Detection-Two-stage method

- **R-CNN**

Combine efficient region proposals (like Selective Search) with powerful CNN classifiers.

The Strategy:

1. Propose: Use Selective Search to generate $\sim 2,000$ candidate regions.
2. Classify: Warp each region and process it with a CNN (e.g., AlexNet) to extract features.
3. Predict: Use the features to classify the object and refine the bounding box.

Shifts object detection from a slow, impractical process to a viable, high-accuracy method.



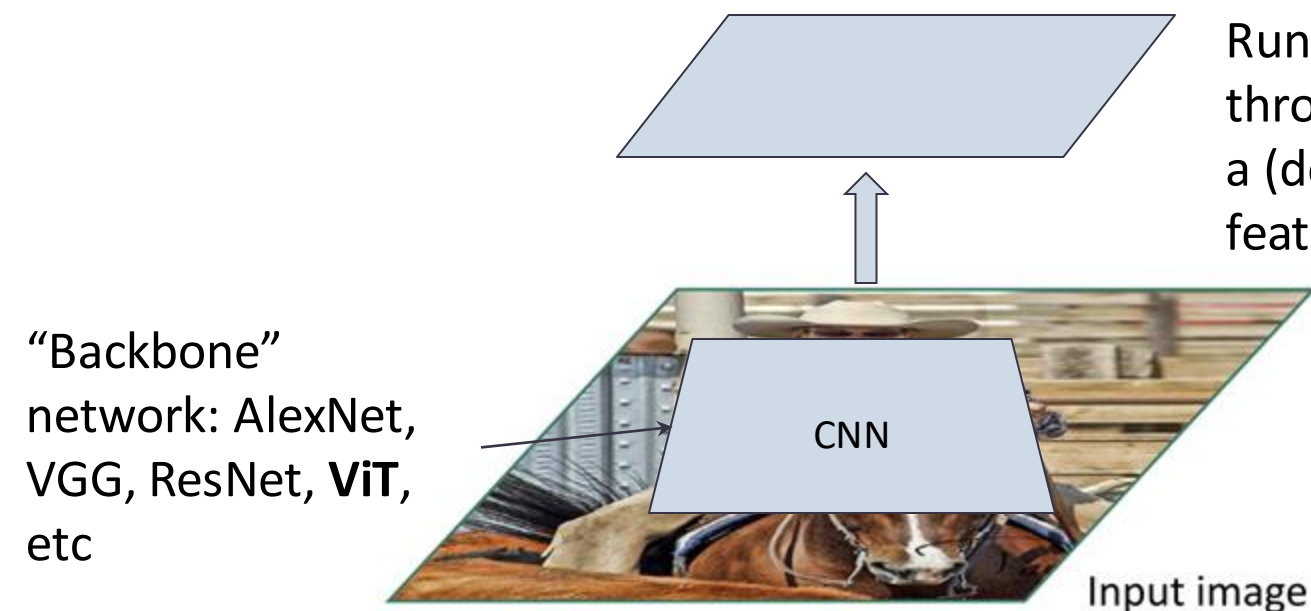
Input image

Recap: Object Detection-Two-stage method

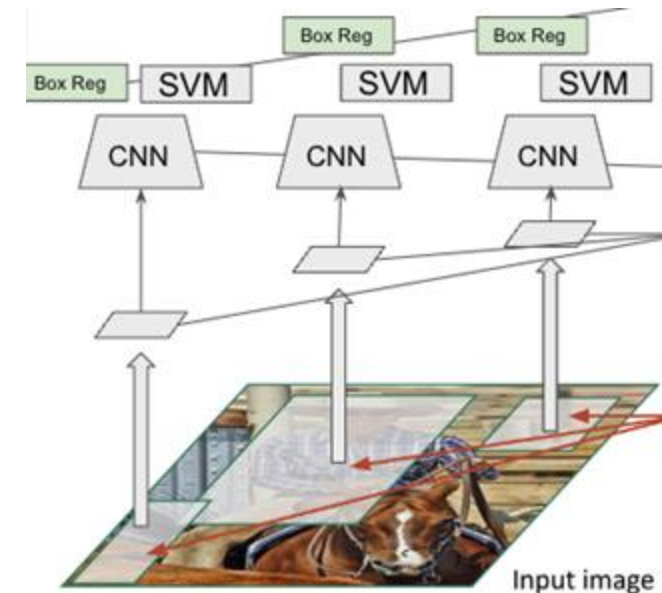
- Fast R-CNN

Key processes:

1. Run the *entire image* through a CNN once to get a feature map.
2. Project region proposals onto the feature map (RoI Projection).
3. Use RoI Pooling to get fixed-size features from each proposal.



Slow R-CNN



Recap: Object Detection-Two-stage method

- **R-CNN vs. Fast R-CNN**

R-CNN's Limitation: Redundant Feature Extraction

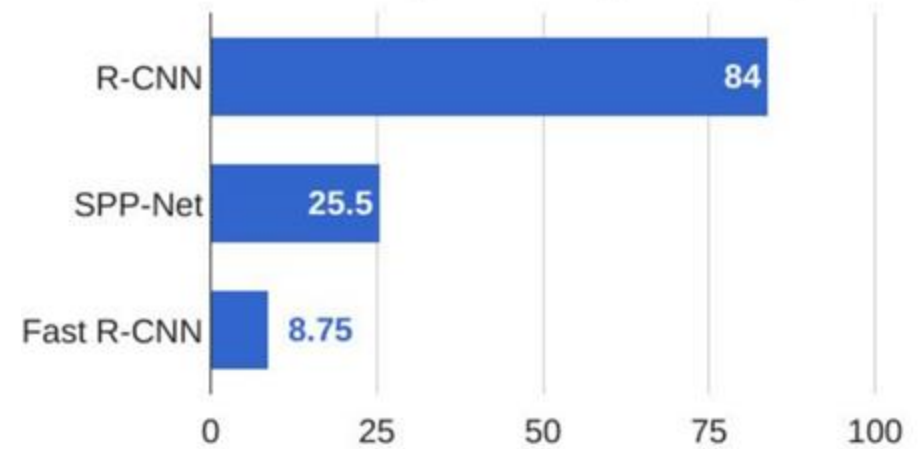
Processing each of the ~2,000 region proposals through the CNN separately is highly inefficient.

This leads to long training times and slow inference.

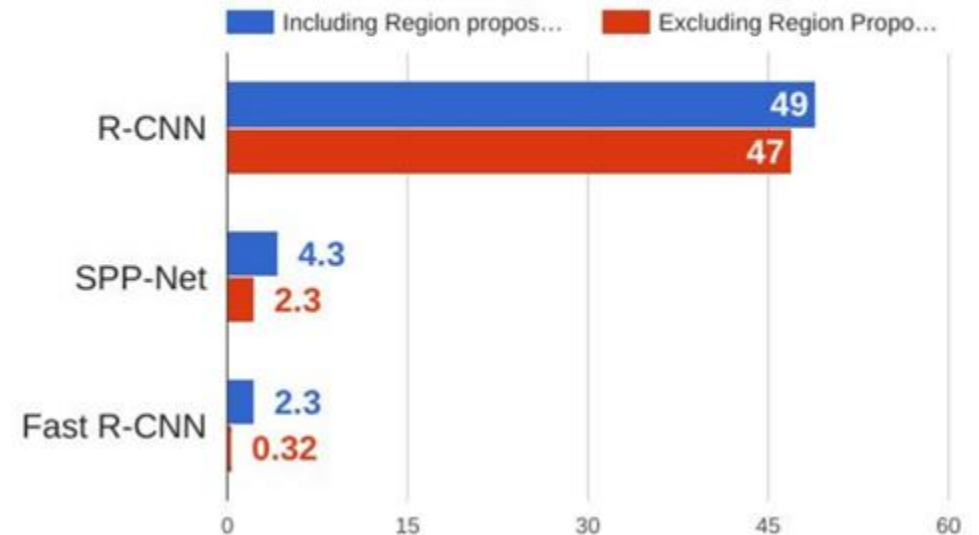
Performance Analysis (See Chart):

- Region Proposal (e.g., Selective Search): ~2 seconds.
- CNN Feature Extraction & Classification: ~47 seconds (the main bottleneck).
- Significant overlap exists between proposed regions. We can share computations by running the entire image through the CNN once.
- Fast R-CNN introduces a more efficient architecture that shares convolutional features across all proposals.

Training time (Hours)



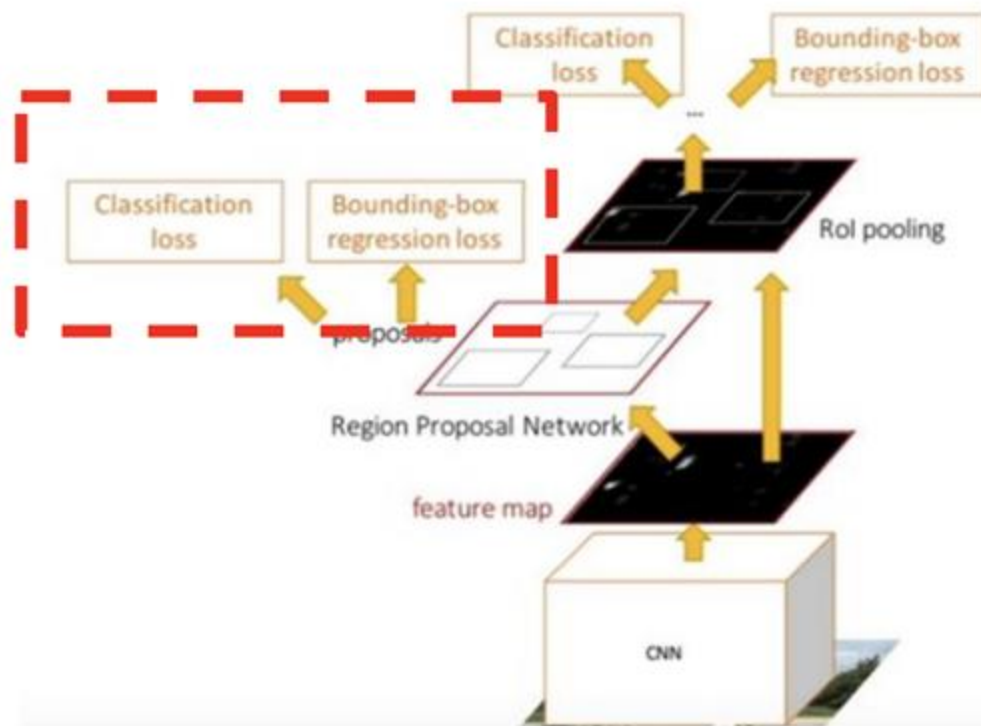
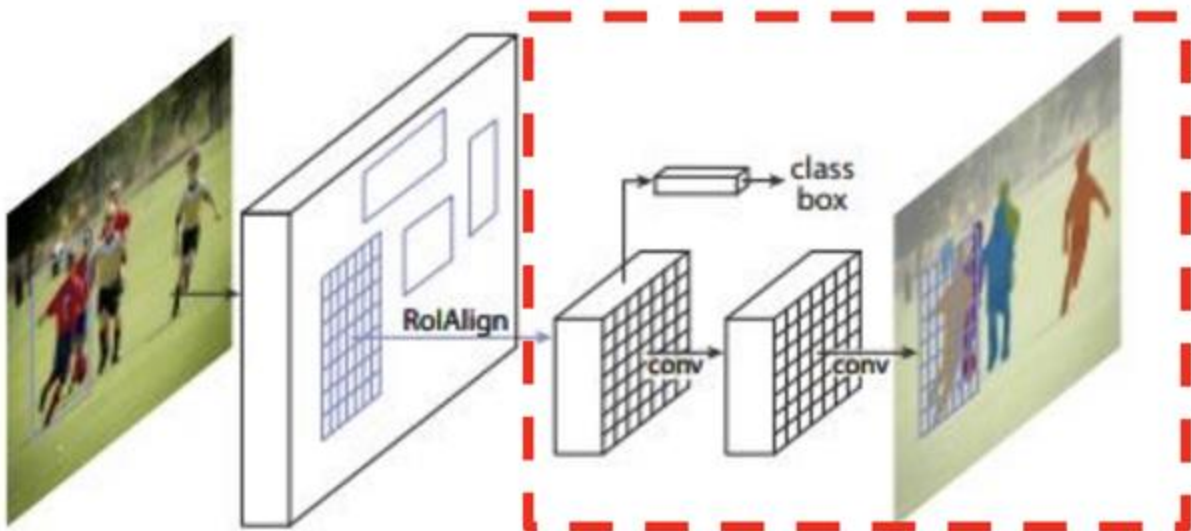
Test time (seconds)



Recap: Object Detection-Two-stage method

- Mask R-CNN

- Mask R-CNN adopts the same two-stage procedure with the identical RPN first stage
- Mask R-CNN outputs a **binary mask** for each RoI for each class in parallel to predicting the class and box offset in the second stage



Recap: Object Detection - Evaluation Metric

•Detection Correctness (IoU Threshold):

- A predicted bounding box is a **True Positive (TP)** if its Intersection-over-Union (IoU) with a ground-truth box exceeds a threshold (typically > 0.5).
- Otherwise, it is a **False Positive (FP)**.

•Evaluation Metrics:

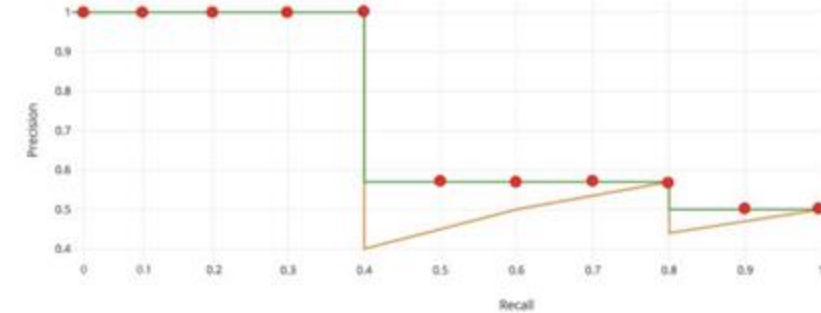
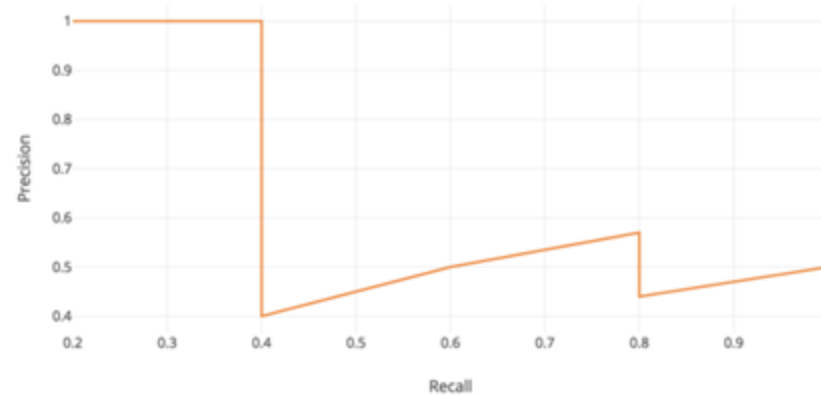
- **Precision:** Of all boxes predicted, how many are correct?
- **Recall:** Of all ground-truth objects, how many did we find?
- **The Precision-Recall Trade-off:**
 - Predictions are ranked by confidence score.
 - As we include more predictions (lower confidence threshold):
 - **Recall Increases** (we find more true objects).
 - **Precision Decreases** (we also include more false alarms).

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0

Recap: Object Detection - Evaluation Metric

Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0



Summarize the entire Precision-Recall curve into one number to compare models easily.

The Method: Average Precision (AP):

- Plot the Precision-Recall curve as the confidence threshold varies.
- Sample the curve at multiple recall levels (e.g., from 0.0 to 1.0).
- At each recall level, use the **maximum precision** achieved at that recall or higher.
- The **AP** is the average of these maximum precision values across all recall levels.

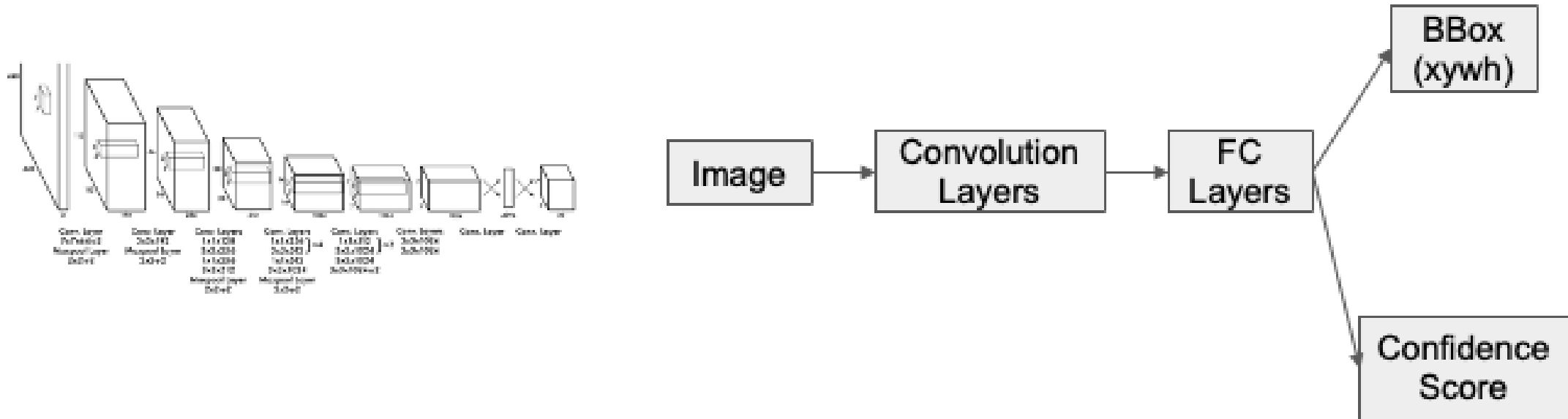
From AP to mAP:

- **AP** evaluates performance on a **single object class**.
- **mAP (Mean Average Precision)** is the average of AP scores across **all object classes** (e.g., 60 classes in a dataset), providing one overall performance score.

Recap: Object Detection - One-stage Model

Key concept: Frame object detection as a single regression problem from image pixels to bounding box coordinates and class probabilities.

- **YOLO**
 - an end-to-end (image $\rightarrow$ 2d-bbox) regression problem (“unified detection”)
 - one-stage detector

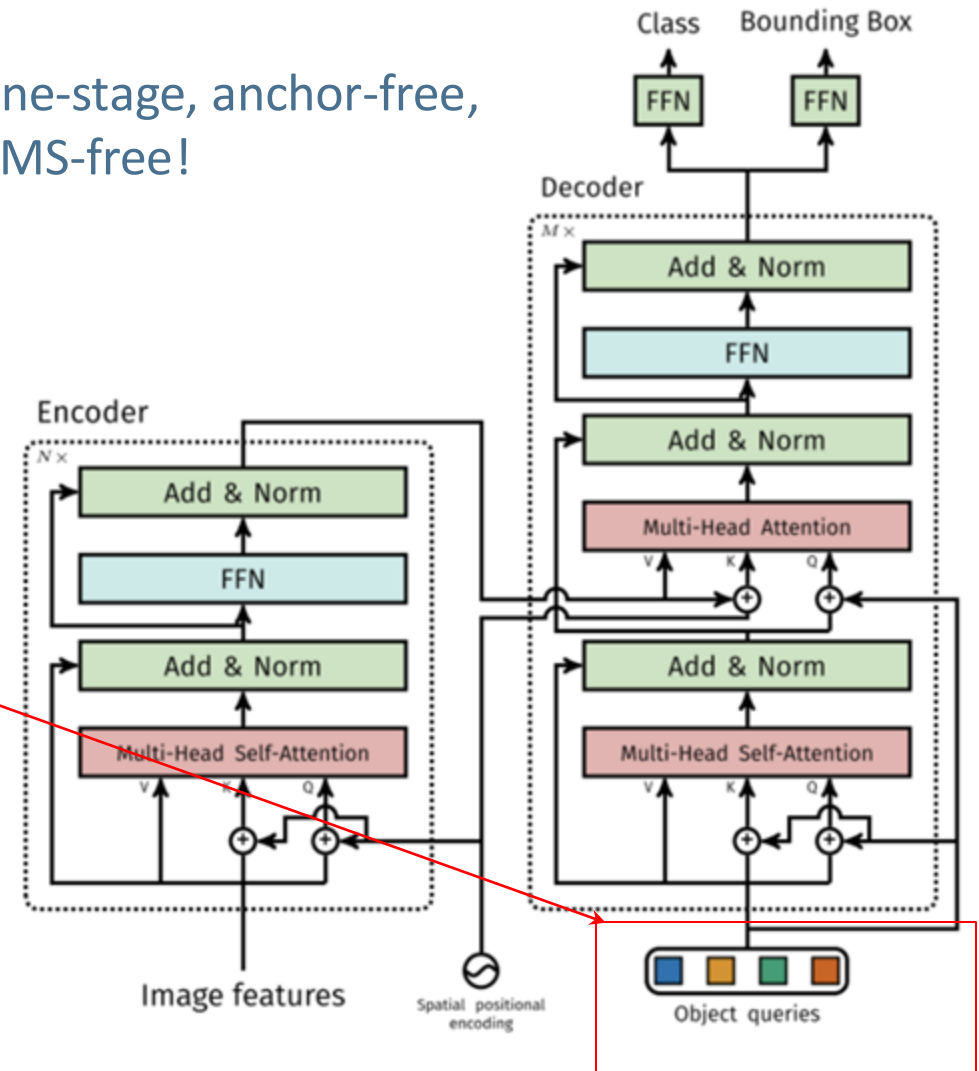


Recap: Object Detection - Transformer

DETR: End-to-End Object Detection with Transformers

- The DETection TRansformer (DETR) predicts all objects at once, and is trained end-to-end and **abandons the anchor boxes** and **non-maximum suppression** during testing
- DETR predicts a **fixed-size set of N predictions in a single pass**. N is chosen to be significantly larger than the typical number of objects (100 queries, etc.)
- During training, the predictions are online assigned to different ground truth boxes with a bipartite matching algorithm (Hungarian)

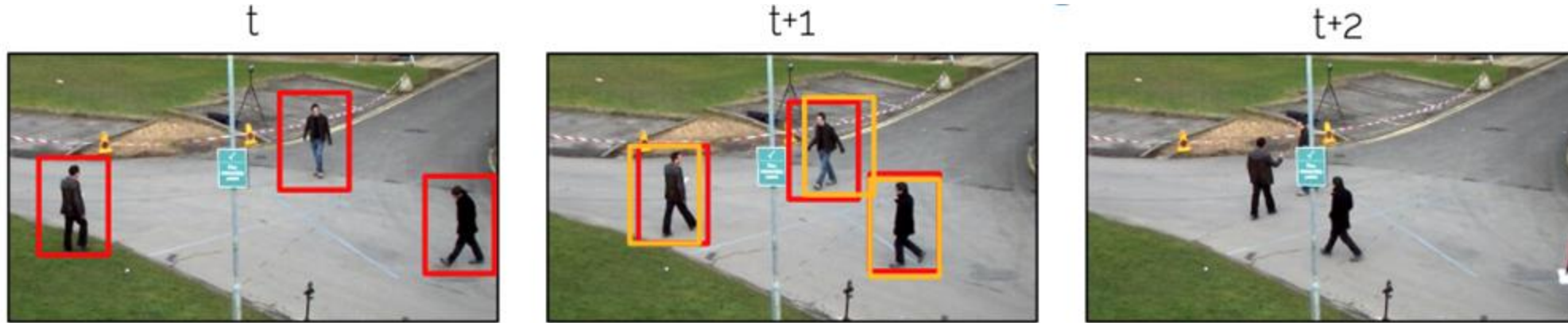
One-stage, anchor-free, NMS-free!



Recap: Object Tracking

- Problem
 - Tracking the same objects/features across multiple video frames
- Types
 - **Tracking without detection**
 - Manually given a bounding box, then track the box
 - Single Object Tracking (SOT, a.k.a. VOT)
 - Traditional vision method based and deep learning based (SiamRPN, CVPR 2019)
 - Not very useful (See DJI active track)
 - **Tracking-by-detection (our focus)**
 - Multi-Object Tracking (MOT)
 - Online tracking & offline tracking
 - Single camera
 - Multi-camera
- **Transition from Detection to Tracking**
 - Detection gives you (bbox, class) per frame.
 - Tracking answers: "Is this object in frame t the same as that object in frame $t+1$?" -> Assigns an ID.

Recap: Object Tracking



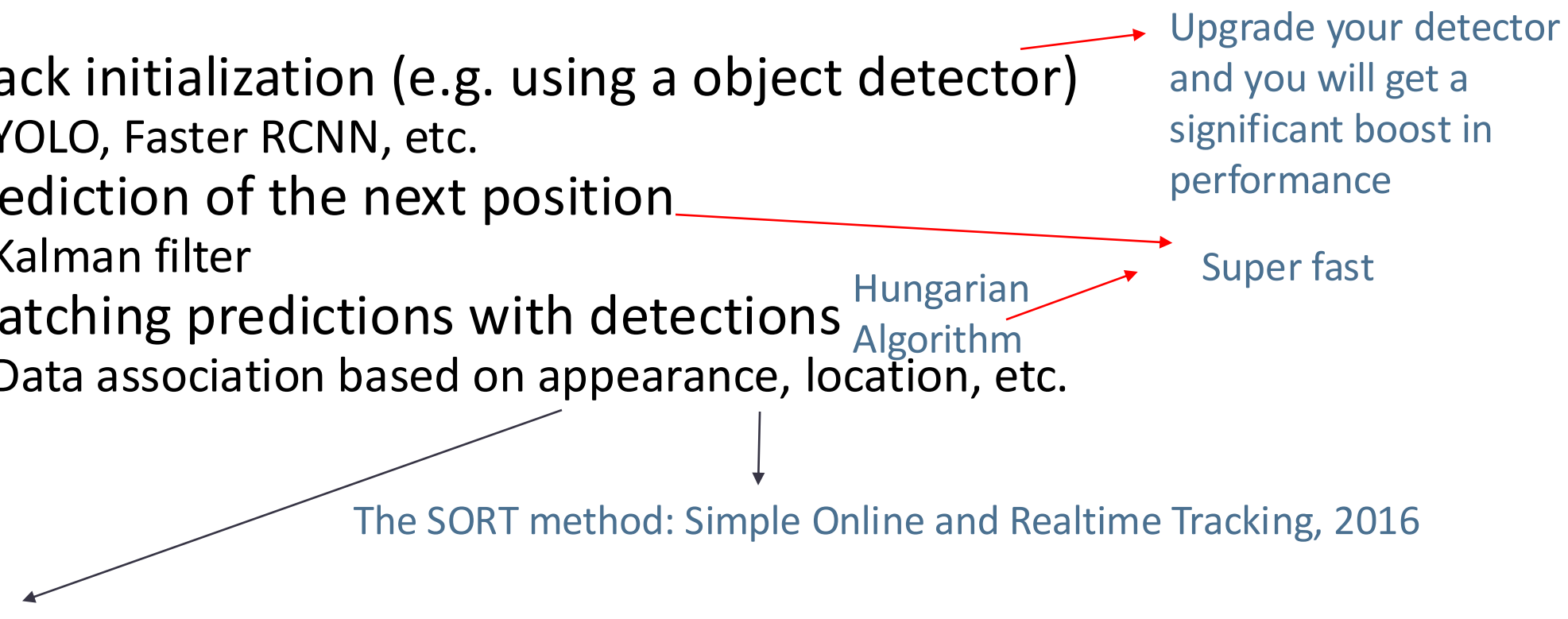
1. Track initialization (e.g. using a object detector)
2. Prediction of the next position (motion model)
3. **Matching predictions with detections**
Data association based on appearance, location, etc.

Data Association

- Given each estimated track state (potential target box locations)
 - Assign tracks with new boxes (new observations)
 - Which is a discrete combinatorial optimization problem

Recap: Object Tracking - Online Tracking

Summary

- 1. Track initialization (e.g. using a object detector)
 - YOLO, Faster RCNN, etc.
 - 2. Prediction of the next position
 - Kalman filter
 - 3. Matching predictions with detections
 - Data association based on appearance, location, etc.
- Hungarian Algorithm
- Upgrade your detector and you will get a significant boost in performance
- Super fast
- 

The SORT method: Simple Online and Realtime Tracking, 2016

The DeepSORT method: Simple Online and Realtime Tracking with a Deep Association Metric, 2017

Test Time

Use Canvas

10 mins 5 questions

Pytorch Basics

Linear Classifier

- The most simple practical classifier
 - W is model weight

$$f(x_i; W) = Wx_i$$



x_i [32, 32, 3]



Dog?
Cat?
Deer?
Ship?

Pytorch Basics

Linear Classifier

- The most simple practical classifier
 - Suppose we have 10 classes
 - **N image to test**; Matrix multiplication is very fast



x_i [N, 32, 32, 3] $\rightarrow$ [3072xN]

[10x3072] x [3072xN]

$$f(x_i; W) = Wx_i$$

[10xN]

Dog: 3.9
Cat: 1.4
Deer: -0.3
Ship: -2.2
...

Pytorch Basics

Linear Classifier

- The most simple practical classifier
 - Suppose we have 10 classes
 - **N image to test**; Matrix multiplication is very fast



x_i [N, 32, 32, 3] $\rightarrow$ [3072xN]

[10x3072] x [3072xN]

$$f(x_i; W) = W x_i$$

[10xN]

Dog: 3.9
Cat: 1.4
Deer: -0.3
Ship: -2.2
...

How to find W?

How to find W - Loss Function

- What is a loss function?
 - An optimization problem (i.e., find a model) is to minimize a loss function
 - Loss function tells **how bad our current model is**
 - We design loss function to find the optimal model weight W

Pytorch Basics

Loss Function

- Loss function tells how bad our current model is
- Given a dataset:

$$\{(x_i, y_i)\}_{i=1}^N$$

- x_i is image and y_i is label (integer)

Pytorch Basics

Loss Function

- Loss function tells how bad our current model is
- Given a dataset: $\{(x_i, y_i)\}_{i=1}^N$
 - x_i is image and y_i is label (integer)
- Our classifier is represented by: $f(x_i; W) = Wx_i$
- Loss over the dataset is an average of loss over examples

$$L = \frac{1}{N} \sum_i L_i(f(x_i; W), y_i)$$

The image classification problem usually uses cross entropy loss - means the correct class should be close to 1, others 0

Then we can change W according to the Loss - This is called optimization!

Pytorch Basics

Optimization

Dataset: $\{(x_i, y_i)\}_{i=1}^N$

- Gradient descent
 - Step 1: compute the gradient for the loss function w.r.t model weight W :

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

- Step 2: apply the gradient to update model weight:

$$W^{t+1} = W^t - \epsilon_t \nabla_W L$$

Pytorch Basics

Optimization

Dataset: $\{(x_i, y_i)\}_{i=1}^N$

- Gradient descent

- Step 1: compute the gradient for the loss function w.r.t model weight W :

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

Problem: N is huge.
Using all of data points might be tricky when computing a gradient

- Step 2: apply the gradient to update model weight:

$$W^{t+1} = W^t - \epsilon_t \nabla_W L$$

Pytorch Basics

Optimization

Dataset: $\{(x_i, y_i)\}_{i=1}^N$

- Gradient descent

- Step 1: compute the gradient for the loss function w.r.t model weight W :

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

Problem: N is huge
Solution: sample mini-batch,
 $N=32/64/128...$
Hence: stochastic
gradient descent

- Step 2: apply the gradient to update model weight:

$$W^{t+1} = W^t - \epsilon_t \nabla_W L$$

Pytorch Basics

Optimization

- Stochastic gradient descent
 - Iteration, epoch

Batch data into
32/64/128 chunks
(mini-batch)

```
while not converged:
    # Shuffle data
    data = randomize(data)
    # Split data into batches and update each batch individual
    for data_batch in data:
        weights_grad = backpropagation(loss_fun, data_batch , weights)
        # perform parameter update
        weights += - step_size * weights_grad
```

Iteration {

} Epoch

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network
 - Dataset & dataloader
 - Network graph
 - The optimization loop

Check out more at <https://pytorch.org/tutorials/beginner/basics/intro.html>

Example code: <https://github.com/hkust-aiaa2205/homework/blob/main/hw2>

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network

- Dataset & dataloader

1. Create a class of Dataset, with `__init__` function to do the **preprocessing** (compute dataset size, define the data augmentation methods, etc.)
2. `__len__` function: define the number of **samples N** for your dataset
3. `__getitem__(self, index)`: this tells Pytorch how to get **one sample** from your dataset

```
class MyDataset(Dataset):  
    def __init__(self, root, csv_file, transform=None):  
        self.root = root  
        self.transforms = transform  
        self.df = pd.read_csv(csv_file, header=None, skiprows=1)  
        self.classes = sorted(self.df[1].unique())  
  
    def __len__(self):  
        return len(self.df)  
  
    def __getitem__(self, index):  
        vid, label = self.df.iloc[index, :]  
        img_list = os.listdir(os.path.join(self.root, f"{vid}"))  
        img_list = sorted(img_list)  
        img_path = os.path.join(self.root, f"{vid}", img_list[int(len(img_list)/2)])  
  
        img = Image.open(img_path).convert('RGB')  
        if self.transforms is not None:  
            img = self.transforms(img)  
  
        label = self.classes.index(label)  
        return img, label
```

Don't load data into memory in init function if the data is huge. Do it at getitem

Define how to do data augmentation of one sample

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network
 - **Network graph – forward computation**

1. Define a `nn.Module` class.
`__init__` function usually defines all the types of layers you want to use in forward function
2. Define **the forward function given input x**: here you provide the compute graph for PyTorch, from `x` to `y_pred`

Could be as simple as this one-liner

```
y_pred = a + b * x + c * x ** 2 + d * x ** 3
```

```
class Net(nn.Module):
    def __init__(self, dropout_prob=0.5):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 128, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(128)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(128, 256, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(256)
        self.fc1 = nn.Linear(256 * 56 * 56, 64)
        self.fc2 = nn.Linear(64, 10)
        self.dropout_prob = dropout_prob

    def forward(self, x):
        x = self.bn1(self.pool(F.relu(self.conv1(x))))
        x = self.bn2(self.pool(F.relu(self.conv2(x))))
        x = x.view(-1, 256 * 56 * 56)
        x = F.dropout(F.relu(self.fc1(x)), p=self.dropout_prob)
        x = self.fc2(x)
        return x
```

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network
 - The optimization loop - Backpropagation

1. In the main training script, we first wrap our dataset with dataloader, which tells Pytorch how to take **mini-batches** from your dataset
2. And choose the optimizer to use. SGD, Adam are commonly used.

```
trainval_dataset = MyDataset("/video_frames_30fpv_320p", "/trainval.csv", transform)
train_data, val_data = train_test_split(trainval_dataset, test_size=0.2, random_state=0)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)

net = Net().to(device)

optimizer = torch.optim.SGD(net.parameters(), lr=0.001, momentum=0.9, weight_decay=0.01)
```

See here the batch size is **32 sample**. Remember previously we have defined `__getitem__` in dataset, which tells pytorch how to get **one sample**. So dataloader will efficiently parallel this

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network
 - **The optimization loop**

3. Now into the optimization loop, starting from **one iteration (using one mini-batch)** in **one epoch**

4. Compute the forward pass and get `y_pred`

5. Compute the loss

6. Backward pass to compute the gradients

7. **Update the model!**

```
for epoch in range(50):  
  
    for i, (inputs, labels) in enumerate(train_loader, 0):  
        y_pred = net(inputs)  
        loss = loss_function(y_pred, labels)  
        optimizer.zero_grad() #zero out all the gradients before  
        loss.backward() # PyTorch's Autograd will compute the backprop gradients for you  
        optimizer.step() # this is when the weights in net() are updated
```

https://pytorch.org/tutorials/beginner/pytorch_with_examples.html#pytorch-optim

Pytorch Basics

- Key ingredients in using PyTorch for training a neural network

- Dataset & dataloader
- Network graph
- The optimization loop



These are the same for object detection training and reinforcement learning training!

For RL (PPO), the dataset will be constructed on the fly with episodes collected from simulation

Check out more at <https://pytorch.org/tutorials/beginner/basics/intro.html>

Example code: <https://github.com/hkust-aiaa2205/homework/blob/main/hw2>

Object Detection Training and Testing

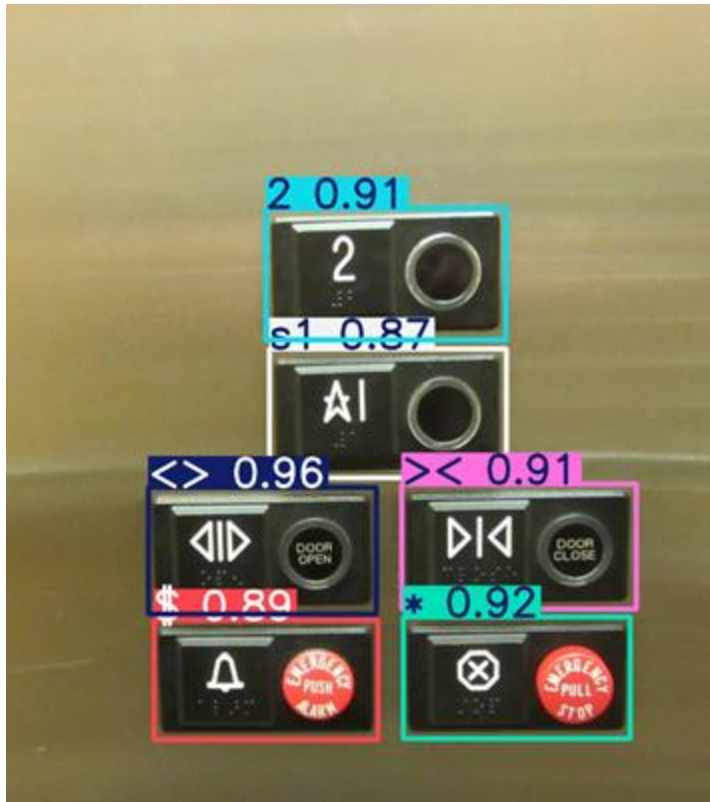
Running YOLO for Project2

- This tutorial will use the following tools
 - An open-source YOLO wrapper - ultralytics
 - <https://docs.ultralytics.com/models/yolov9/>
 - Should be easy to install
 - pip install ultralytics
 - Opencv - to read images
 - pip install opencv-python
- Code base
 - https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py
 - This is for “Inference” using your laptop camera. We learn this in Lecture 8
 - Project 2 will be about ego-view object detection
 - Now we go through training and testing of object detection models

Object Detection Training and Testing

Embodied AI Case Study

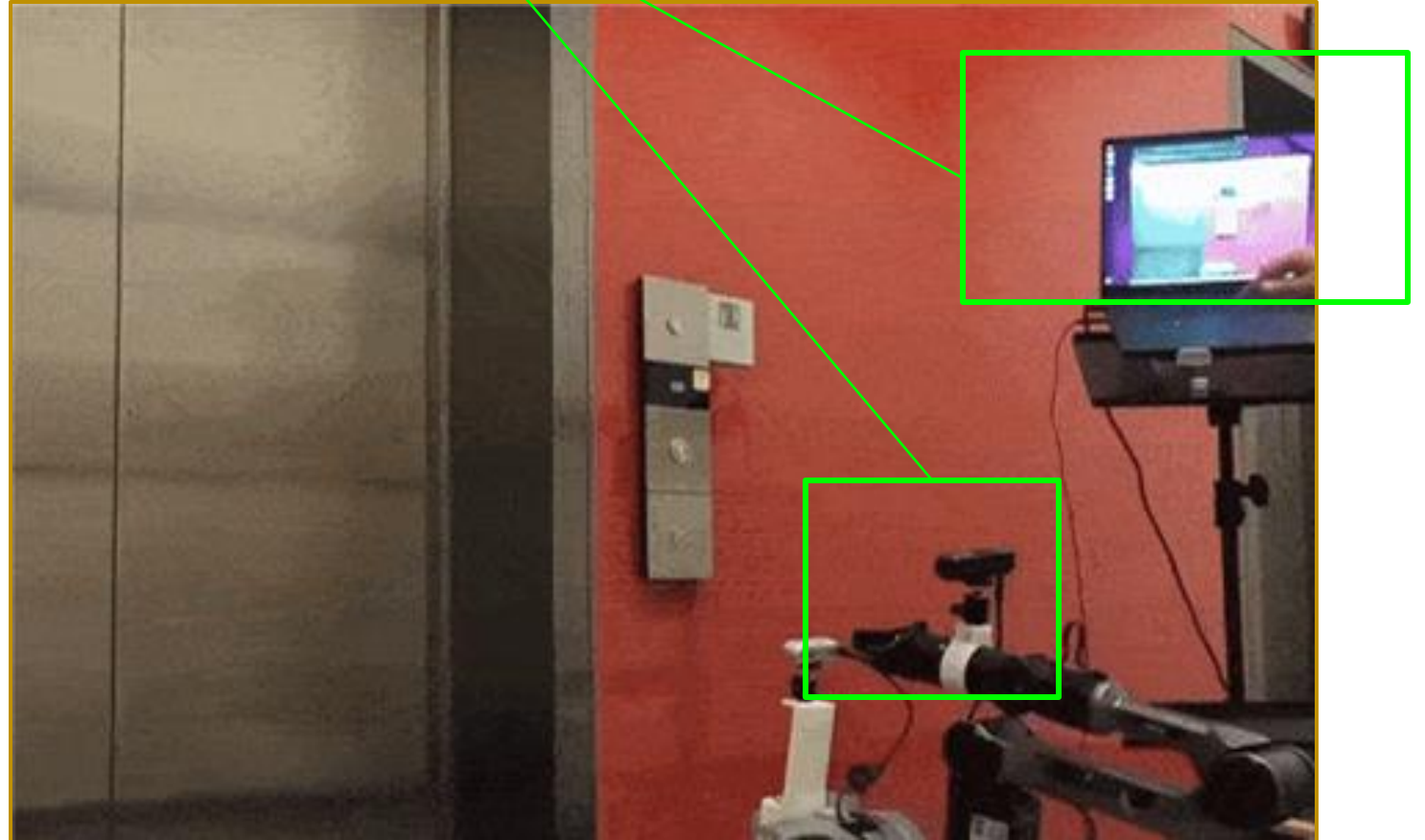
- Suppose we want to build a robot that can use the elevators - it needs to detect certain buttons and push them! Like the following demo:



Object Detection Training and Testing

Embodied AI Case Study

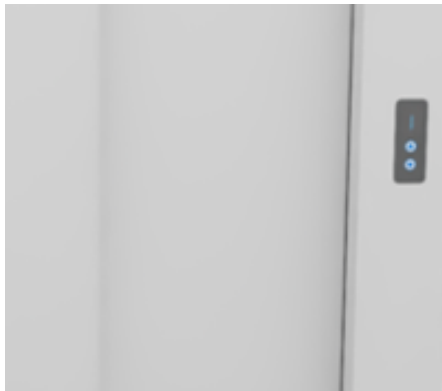
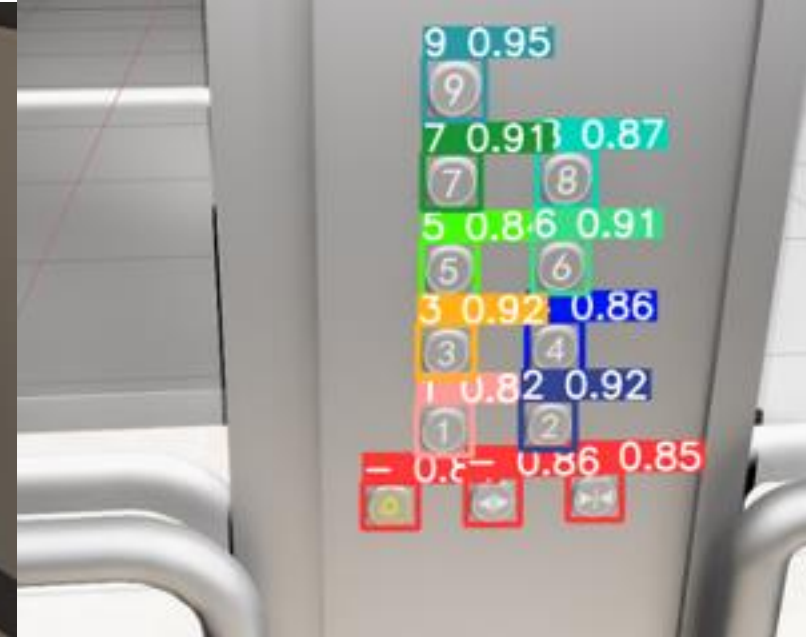
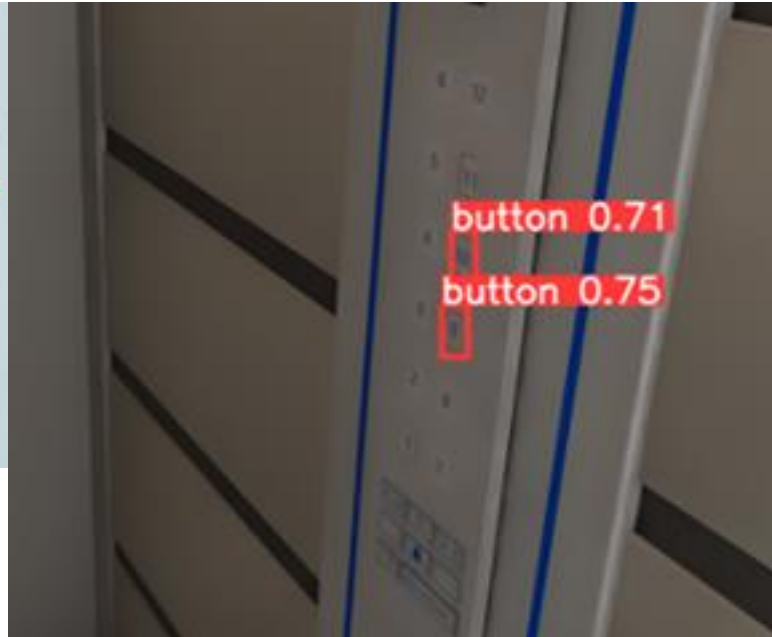
- The work flow: Depth camera on robot arm **detects buttons** - then **push** it!
 - Easy! We will learn the push part later in this semester



Object Detection Training and Testing

Embodied AI Case Study

- Not so fast! Detecting elevator buttons are not easy!
 - The button classes are not in the COCO dataset. And you have to account for many variants. Button types, lighting, reflections, etc.



Pretrained COCO model
does not work

Many button types to cover! 1-9,
open, close, etc.

Object Detection Training and Testing

Embodied AI Case Study

- Not so fast! Detecting elevator buttons are not easy!
 - VLM also does not work well! (this is a 22B model example)



Is it an elevator button, and what button it is if so? Return in JSON format: { "is_button": true or false, "button_type": button_type or null }, where "button_type" can be a floor button ("floor-X"), a open door button ("open"), a close door button ("close"), an alarm button ("alarm"), or something else ("other").

```
{
  "is_button": true,
  "button_type": "floor-7"
}
```



Object Detection Training and Testing

Embodied AI Case Study

- The work flow: Depth camera on robot arm **detects buttons** - then push it!
 - No pretrained model
 - We will train/fine-tune one ourselves
 - Getting the dataset: label it yourself
 - Training/finetuning
 - Testing (so you know how your model performs)

Object Detection Training and Testing

Embodied AI Case Study

- **Getting the dataset** - Elevator buttons - what we need
 - The object bounding boxes and the object classes - a lot of these



The label file corresponding to the above image contains 2 persons (class 0) and a tie (class 27):

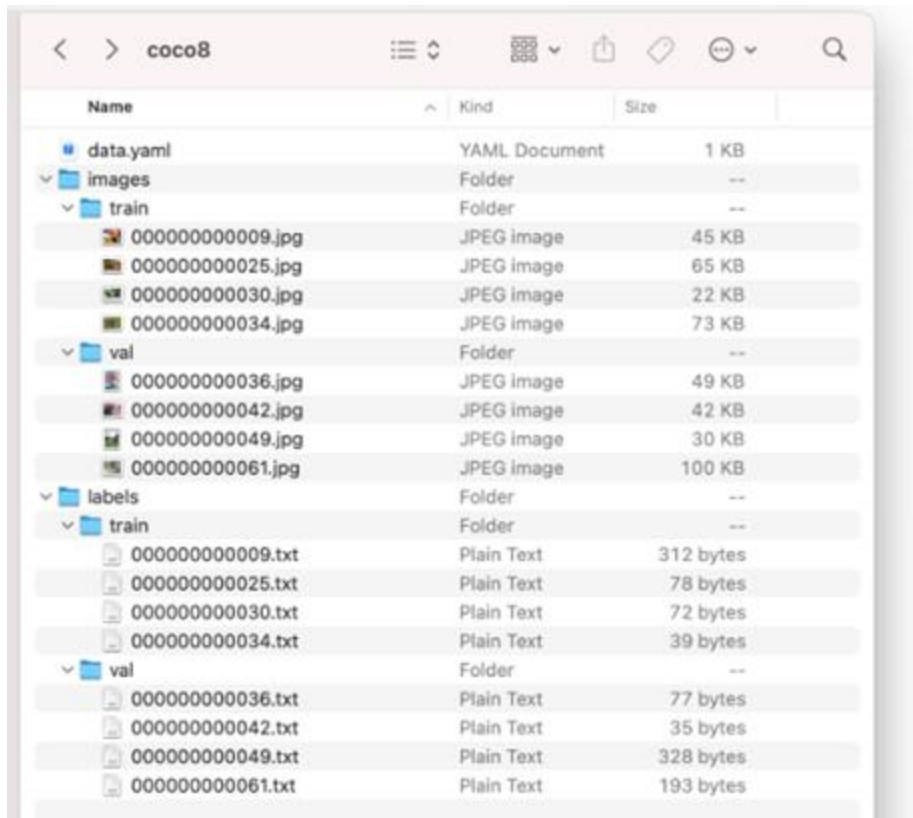
```
zidane.txt v
0 0.481719 0.634028 0.690625 0.713278
0 0.741094 0.524306 0.314750 0.933389
27 0.364844 0.795833 0.078125 0.400000
```

- For labels, remember to check:
1. xxyy or xywh?
 2. Relative box or absolute box?

Object Detection Training and Testing

Embodied AI Case Study

- **Getting the dataset** - Elevator buttons - what we need
 - The object bounding boxes and the object classes - File structure



See here, we should at least have training set and validation set. Each have images, and the corresponding labels (plain text). Ideally you should also have a test set.

What are the purposes of train/val/test?

Model performance parameters hyper-parameters?

Object Detection Training and Testing

Embodied AI Case Study

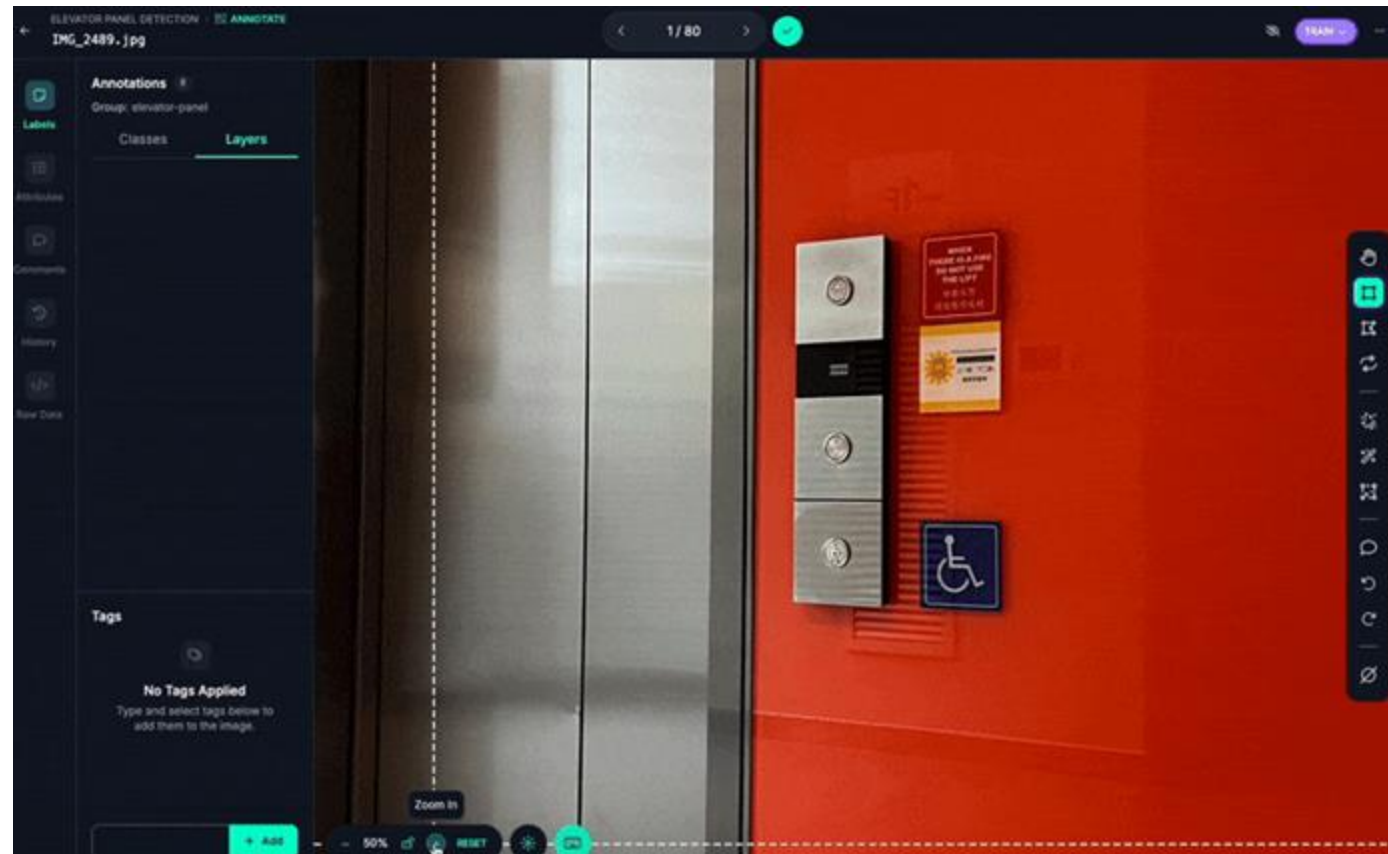
- **Getting the dataset** - Elevator buttons - The labeling tools
 - You can use [Roboflow](#) - it's free and it will export Ultralytics format for YOLO training
 - There are many other labeling tools - they allow you to easily draw bounding boxes (annotate)
 - Step 1: collect a lot of real-world elevator panel images from the robot perspective (we got 200)



Object Detection Training and Testing

Embodied AI Case Study

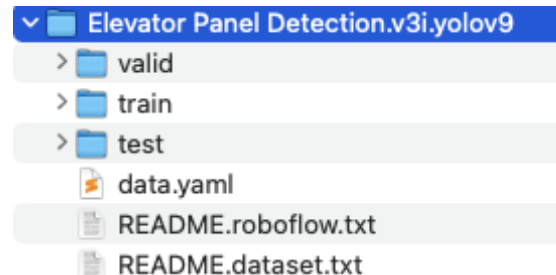
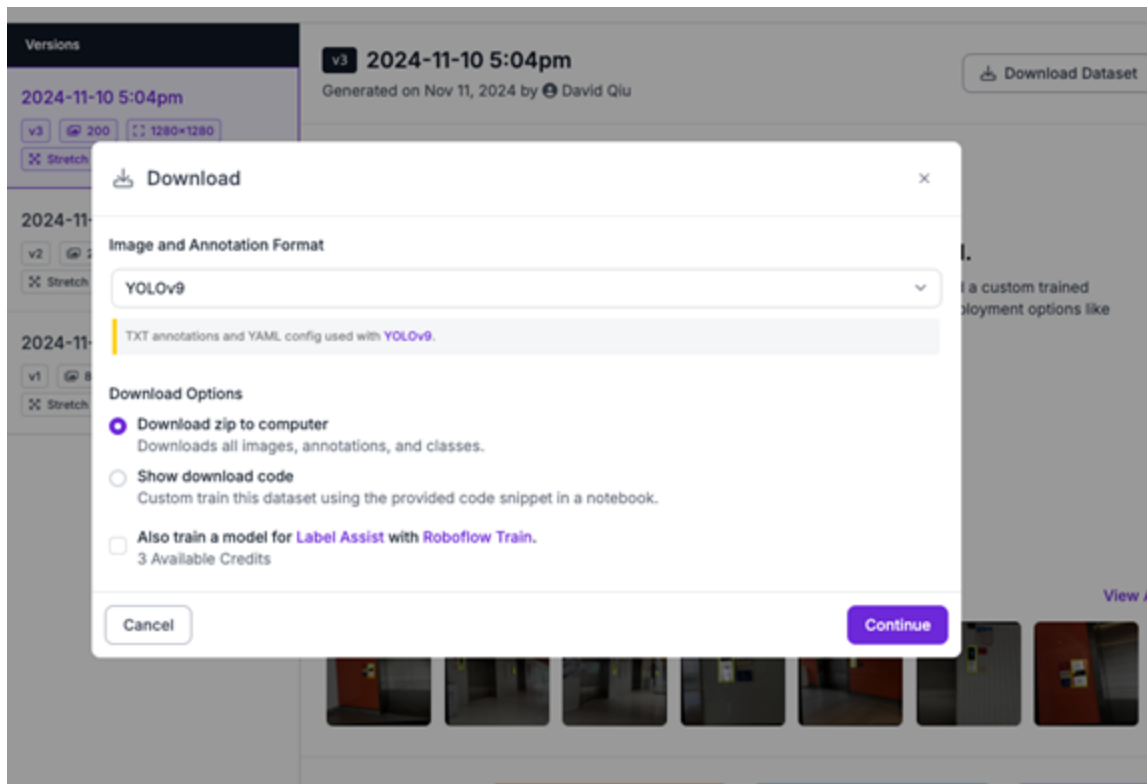
- **Getting the dataset** - Elevator buttons - The labeling tools - [Roboflow](#)
 - Step 2: for each of your image, draw bounding boxes + label classes
- Bounding boxes are the easiest to annotate. Roboflow can also do segmentation, etc.



Object Detection Training and Testing

Embodied AI Case Study

- **Getting the dataset** - Elevator buttons - The labeling tools - [Roboflow](#)
 - Step 3: Done! Split your dataset into train/val and export the dataset



These are the exported files. YOLO/ultralytics will use the data.yaml file

Object Detection Training and Testing

Embodied AI Case Study

- Training/finetuning

```
from ultralytics import YOLO
```

```
# Load a model
```

```
model = YOLO('yolo9n.yaml') # build a new model from scratch
```

```
model = YOLO('yolo9n.pt') # load a pretrained model (recommended for training)
```

```
# Train/Finetune the model!
```

```
results = model.train(data='data.yaml', epochs=10, batch=16, imgsz=640, lr0=0.001, lrf=0.01)
```

```
# lr0: Initial learning rate (i.e. SGD=1E-2, Adam=1E-3).
```

```
# Adjusting this value is crucial for the optimization process, influencing how rapidly model weights are updated.
```

```
# lr1: Final learning rate as a fraction of the initial rate = (lr0 * lrf),
```

```
# used in conjunction with schedulers to adjust the learning rate over time.
```

```
# for example, cosine decay is a good method: cos_lr=True
```

- Key ingredients in using PyTorch for training

- Dataset & dataloader

- Network graph

- The optimization loop

For more: <https://colab.research.google.com/github/ultralytics/ultralytics/blob/main/examples/tutorial.ipynb>

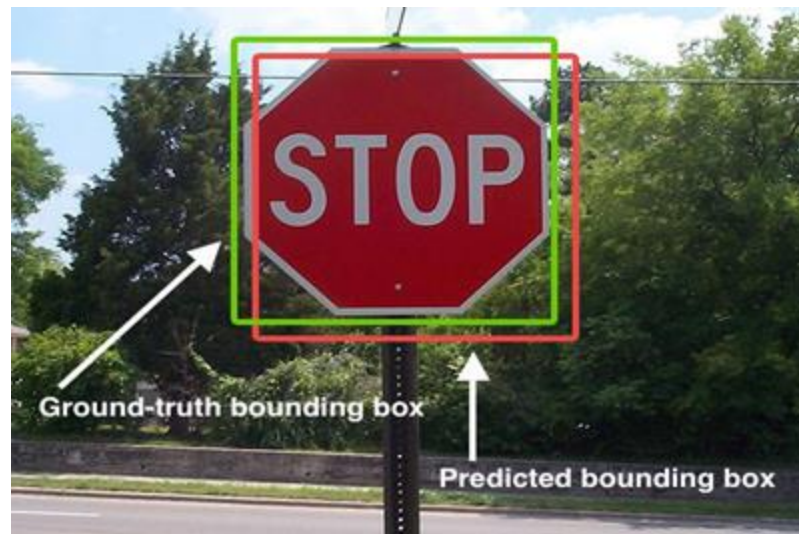
Documentation: <https://docs.ultralytics.com/modes/train/#train-settings>

Object Detection Training and Testing

Embodied AI Case Study

- **Testing** (so you know how your model performs)
 - Remember in L7: we use mean AP to evaluate the object detection model

```
results = model.val() # evaluate model performance on the validation set  
results = model('elevator.jpg') # predict on an image
```



Object Detection Training and Testing

Embodied AI Case Study

- The work flow: Depth camera on robot arm **detects buttons** - then **push** it!
 - See this real-world test results. All buttons are detected correctly

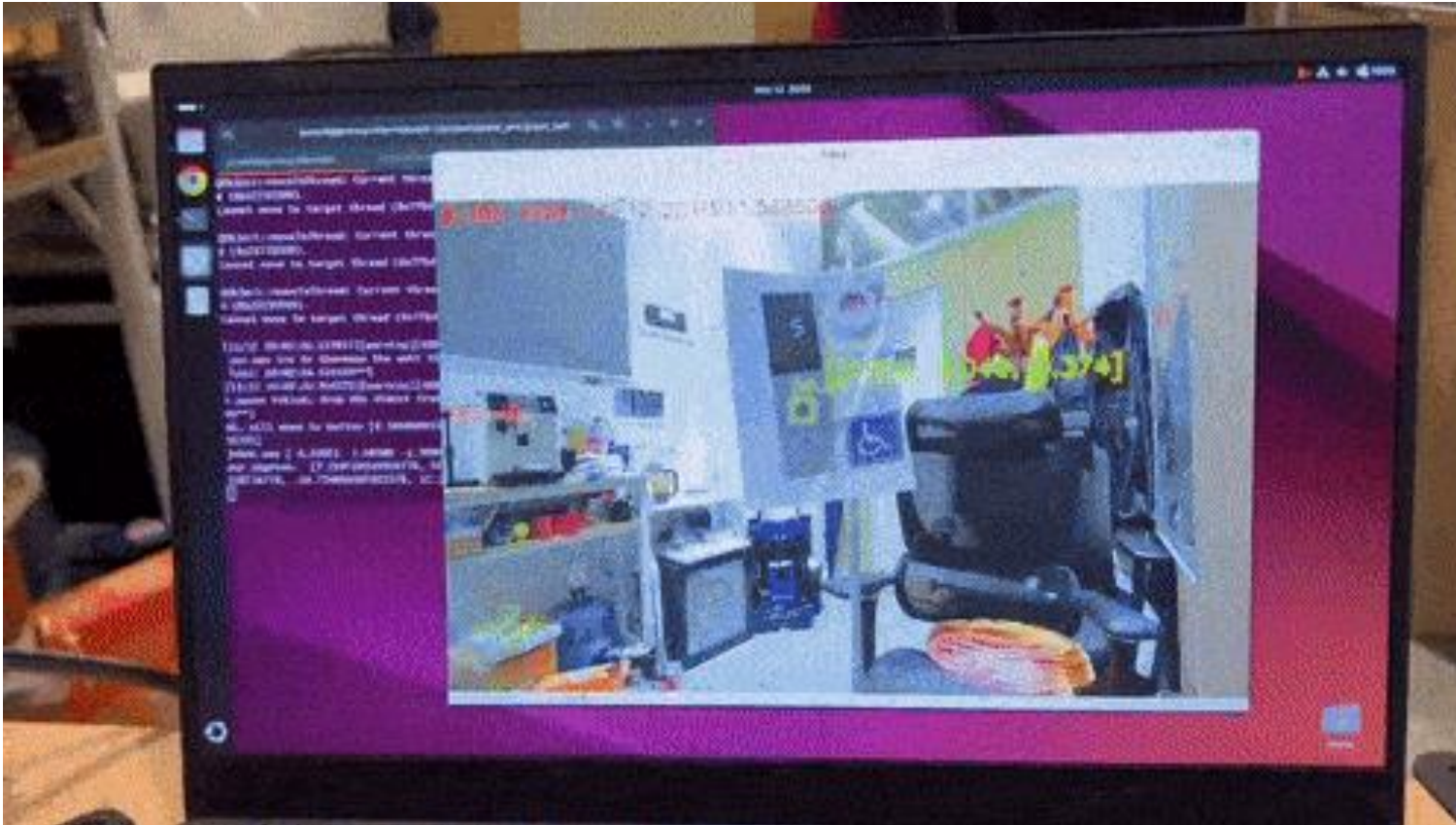
We use "(" for up button and ")" for down button. "<>" for open, "><" for close. "*" for unknown button



Object Detection Training and Testing

Embodied AI Case Study

- The work flow: Depth camera on robot arm **detects buttons** - then **push** it!
 - Remember in Lecture 4, **we need depth camera to convert pixel boxes to 3D!**

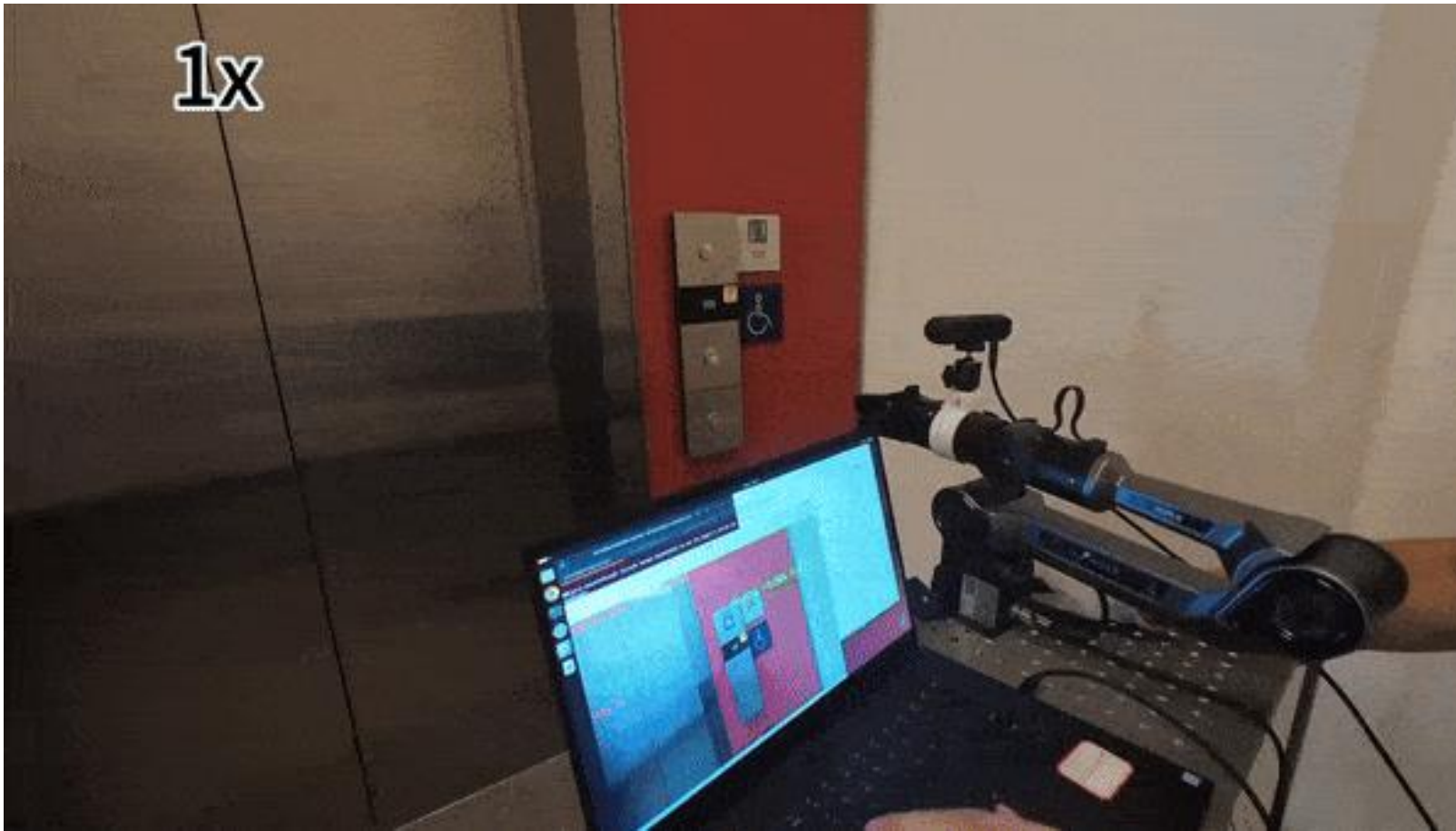


Usually we will do unit test in low-risk environment first. Like this paper button panel

Object Detection Training and Testing

Embodied AI Case Study

- The work flow: Depth camera on robot arm **detects buttons** - then **push** it!
 - Remember in Lecture 4, we need depth camera to convert pixel boxes to 3D!



Then we can try it in the real-world setting!

Project 2 Release

https://peaklab-hkust-gz.github.io/AIAA_4220/EAI-fall-2025/project/

Overview

AIAA 4220 - Introduction to Embodied AI Project 2: Egocentric Object Detection Challenge

Welcome to the second project of AIAA 4220 Introduction to Embodied AI course! This competition challenges you to develop robust object detection algorithms for first-person perspective images.

 **Kaggle Competition:** <https://www.kaggle.com/t/beb9f7424edf46799536b93620e26f2f>

 **How to Join Kaggle Competition:** <https://www.kaggle.com/discussions/general/400680>

Competition Highlights:

- **Dataset:** 13,000 egocentric images from EgoObjects dataset
- **Task:** Detect 10 common object categories in first-person view
- **Framework Freedom:** Use any detection method (YOLO, Faster R-CNN, DETR, etc.)
- **Team Size:** 1-3 students per team
- **Baseline Performance:** mAP@0.5:0.95 of 0.759-0.788 provided (MMDetection)
- **Prize:** Top 3 teams present methods on Dec 3 (4%/2%/1% extra credit)

This is a **learning-focused competition** where methodology explanation and code quality matter as much as leaderboard ranking. Perfect for exploring modern object detection techniques while working with challenging real-world data.

3D Simulator

CARLA

- Introductory video. See what it can offer in 3D simulation - Latest CARLA 0.9.15
- Autonomous driving challenge. See what can be achieved in 3D simulation



3D Simulator

CARLA

- What is CARLA?
 - Unreal Engine based driving simulator for training autonomous vehicle agents
 - Simulates physics and renders models of real world environments and vehicles
 - Uses the OpenDRIVE standard 1.4 to define road networks
 - OpenDRIVE is an open format specification to describe a road network's logic. Its objective is to standardize the logical road description between different driving simulators
 - Provides an API in C++ and Python to control the simulator

3D Simulator

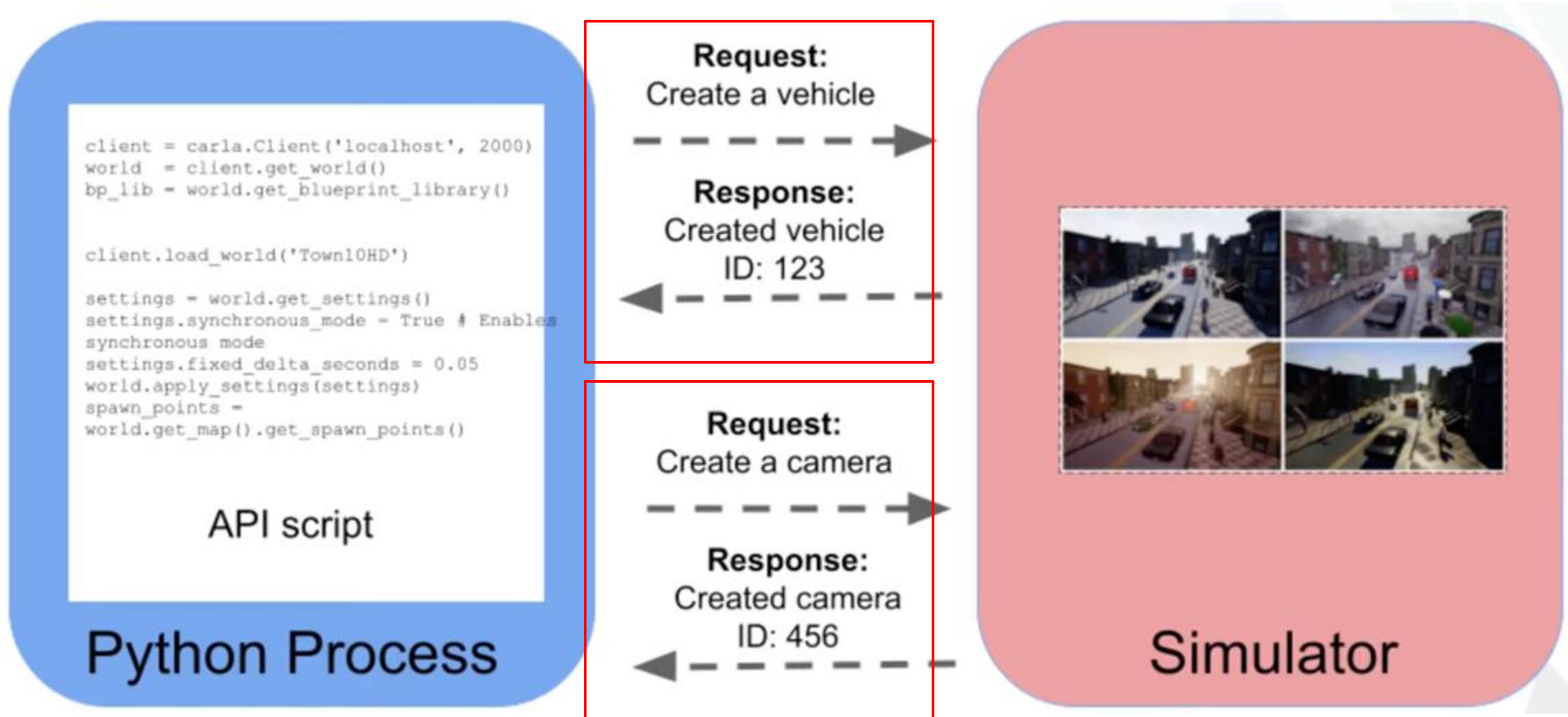
CARLA

- What is CARLA?
 - Integrates with multiple 3rd party applications in autonomous driving and robotics



3D Simulator

CARLA Client / Server Architecture



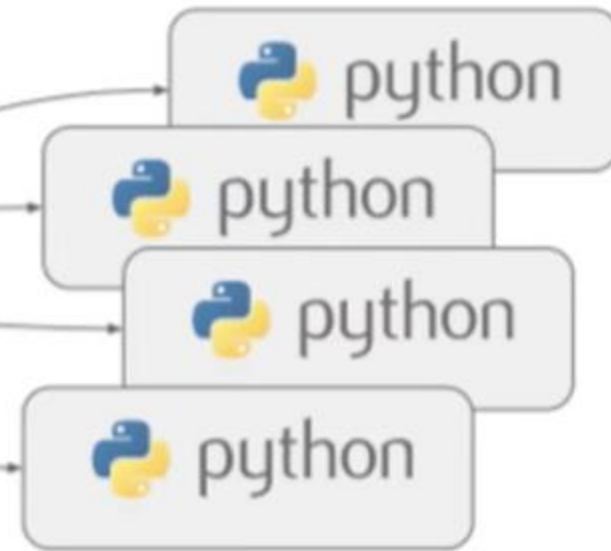
3D Simulator

CARLA Client / Server Architecture

- The server can connect to multiple clients
 - So you can connect to multiple machine learning model processes
 - E.g., a process for pedestrian detection; another for vehicle control



Simulator

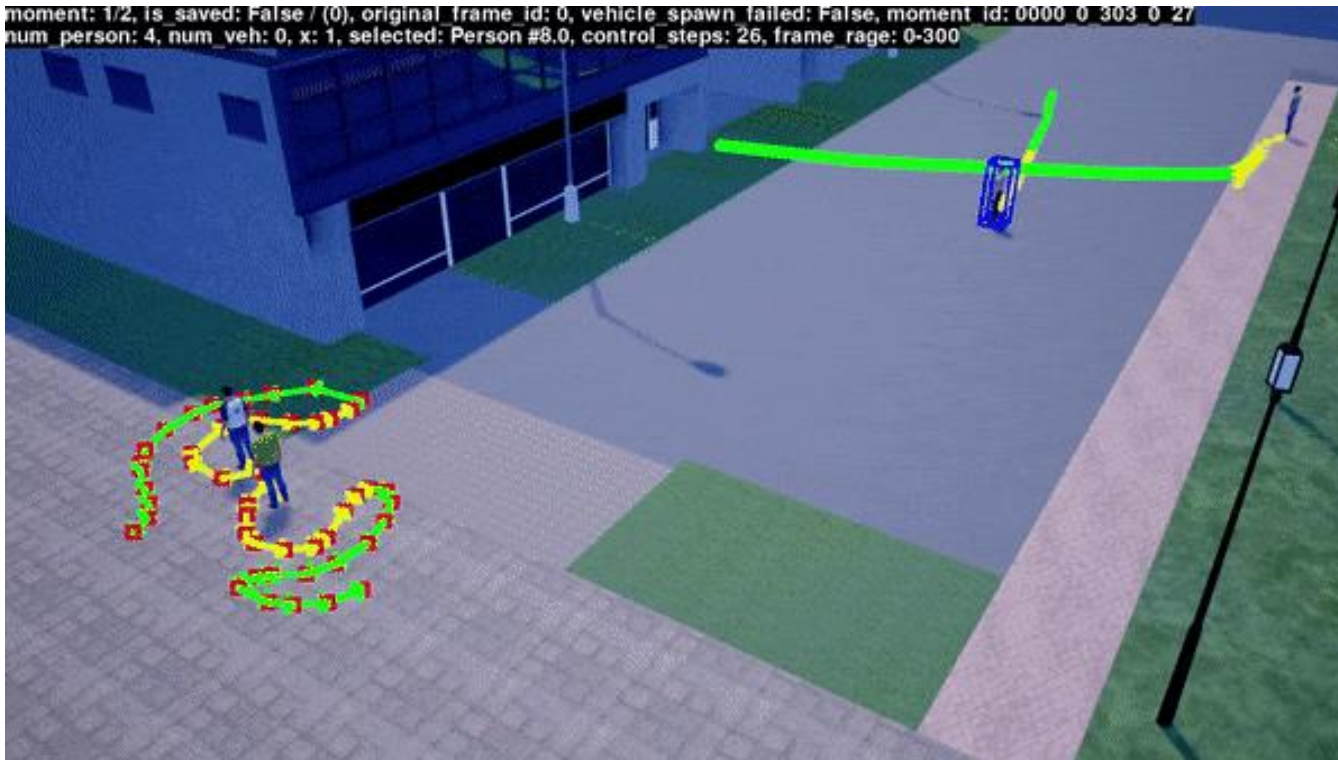


User scripts

3D Simulator

CARLA Client / Server Architecture

- The server can connect to multiple clients
 - One of the client could also be a **spectator** that you can control and move around
 - In this example, you can look around the scene and create agents as you like



3D Simulator

CARLA Core Concepts

- World and Client - the system architecture
 - The world is a Python object representing the simulation. It acts as an abstract layer containing the main methods to spawn actors, change the weather, get the current state of the world, etc. **There is only one world per simulation**
 - **A client runs with an IP and a specific port.** It communicates with the server via network
 - So you can run CARLA on multiple computers! Not just on a single machine
- Actors and blueprints - key dynamic elements in the simulator
 - An actor is anything that plays a role in the simulation
 - Blueprints are already-made actor layouts necessary to **spawn an actor**
- Maps and navigation - static elements you can edit with UE4 editor
- Sensors and data
 - **A sensor is an actor** attached to a parent vehicle
 - The RGB data, etc., will generate from this sensor

3D Simulator

CARLA Synchrony

- Simulation time-step

- There is a difference between real time, and simulation time.
- Computing one simulation step takes some real time
- Time-step
 - time span that went by between those two simulation moments
- the server can take a few milliseconds to compute two steps of a simulation
- the time-step between those two simulation moments can be configured to be, for instance, always a second
- So you can set fixed time-step if you want to see simulated data in real-world time!*

For example, we want the vehicle to move forward a bit in the next step



- Also setting the synchronous mode

```
settings = world.get_settings()
settings.synchronous_mode = True
settings.fixed_delta_seconds = 1.0 / args.video_fps
world.apply_settings(settings)
```

* https://carla.readthedocs.io/en/0.9.14/adv_synchrony_timestep/

* https://github.com/JunweiLiang/Multiverse/blob/master/forking_paths_dataset/code/record_annotation.py

3D Simulator

CARLA Synchrony

- Key concepts and functions
 - Every time the simulator does an update (advances the simulation one step), it sends a message to all the clients. We also call this update a "tick"
 - `world.tick()` tells the simulator to perform a tick (i.e. advance one step). This only has effect in synchronous mode.
 - So if you enable synchronous mode, the simulator is going to halt until you call `world.tick()`

* https://carla.readthedocs.io/en/0.9.14/adv_synchrony_timestep/

* <https://github.com/carla-simulator/carla/issues/3479>

3D Simulator

CARLA Synchrony

- Fixed **delta time** (same concept used in other simulators)
 - The time that will be simulated at each tick
 - The value is calculated dividing 1 second by the desired frame rate.

Fixed Delta Time

	1 second											
5 frames	0.2		0.2		0.2		0.2		0.2		$1/5 = 0.2$	
10 frames	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	$1/10 = 0.1$	
20 frames	.05	.05	.05	.05	.05	.05	.05	.05	.05	.05	.05	$1/20 = 0.05$

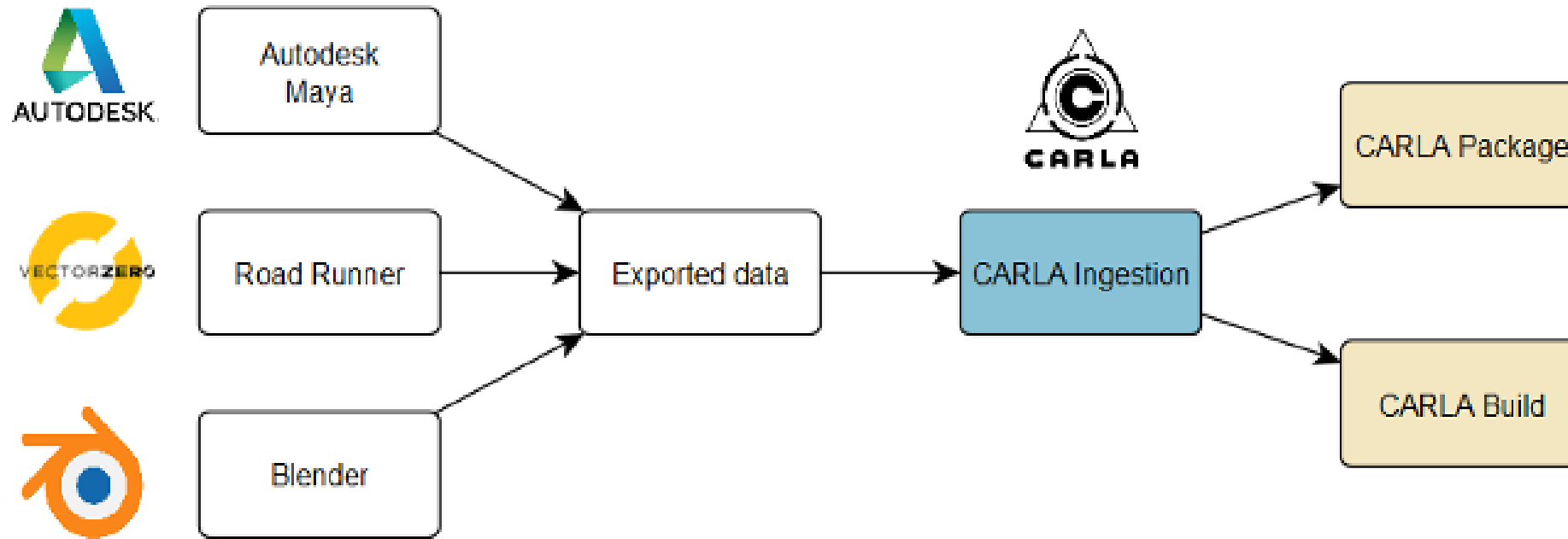
So if you set 5 frames per second, you get one simulation step at each 0.2 second

We use similar idea when controlling real robots like Unitree G1. We set a control frequency of 100 Hz for example, so the while loop will make sure it will run **at least** 0.01 second per loop. So this means the script will run at most 100Hz. The program will run as fast as possible. We **can only control the maximum speed** to run but not the minimum.

3D Simulator

CARLA Data Ingestion

- From any software to CARLA
 - You can also directly edit the map using UE4 and rebuild (“cook”) the map like this*



* https://github.com/JunweiLiang/Multiverse/tree/master/forking_paths_dataset#edit-the-maps

* More on this: <https://drive.google.com/file/d/10uNBAMreKajYimIhwCqSYXjhfVs2bX31/view>

3D Simulator

CARLA Pedestrians

- CARLA provides pedestrians with simple navigation abilities so you could populate your environment
 - use the open source Recast & Detour library*

See here after you spawn the pedestrian actor, you can call the controller to “go_to_location” to make the pedestrian walk

```
client = carla.Client("localhost", 2000)
world = client.get_world()

walker_bp = world.get_blueprint_library().filter("walker.pedestrian.*")
controller_bp = world.get_blueprint_library().find('controller.ai.walker')

trans = carla.Transform()
trans.location = world.get_random_location_from_navigation()
trans.location.z += 1

walker = random.choice(walker_bp)
actor = world.spawn_actor(walker, trans)
world.wait_for_tick()

controller = world.spawn_actor(controller_bp, carla.Transform(), actor)
world.wait_for_tick()

controller.start()
controller.go_to_location(world.get_random_location_from_navigation())
```



*<https://github.com/recastnavigation/recastnavigation>

*More on this: <https://drive.google.com/file/d/1Tsosin7BLP1k558shtbzUdo2ZXVKy5CB/view>

3D Simulator

CARLA Pedestrians

- You can even control the joints of a pedestrian



*https://carla.readthedocs.io/en/latest/tuto_G_pedestrian_bones/
*https://carla.readthedocs.io/en/latest/tuto_G_control_walker_skeletons/

3D Simulator

CARLA

- You can first try to install it and control a car using keyboard
 - A computer with RTX 3060 and above works
- Next you can try to use a machine learning model to control it!
 - Maybe even compete with others internationally on the CARLA Leaderboard!



3D Simulator

Nvidia Isaac Simulator

- The latest 3D simulator for robot learning by Nvidia
- On top of Isaac Sim, Nvidia builds Isaac Lab, with tons of robots and baseline policies you can play with.



3D Simulator

Nvidia Isaac Simulator

- Isaac Sim and Isaac Lab
 - Easy to install. Simple pip/conda install should work with a Ubuntu machine
 - Comes with baseline (locomotion) policy* you can train and then control a robot to move (just like CARLA)



So you can give the robot a keyboard command to move in one direction, it knows how to control the legs and arms to balance itself (tries to)

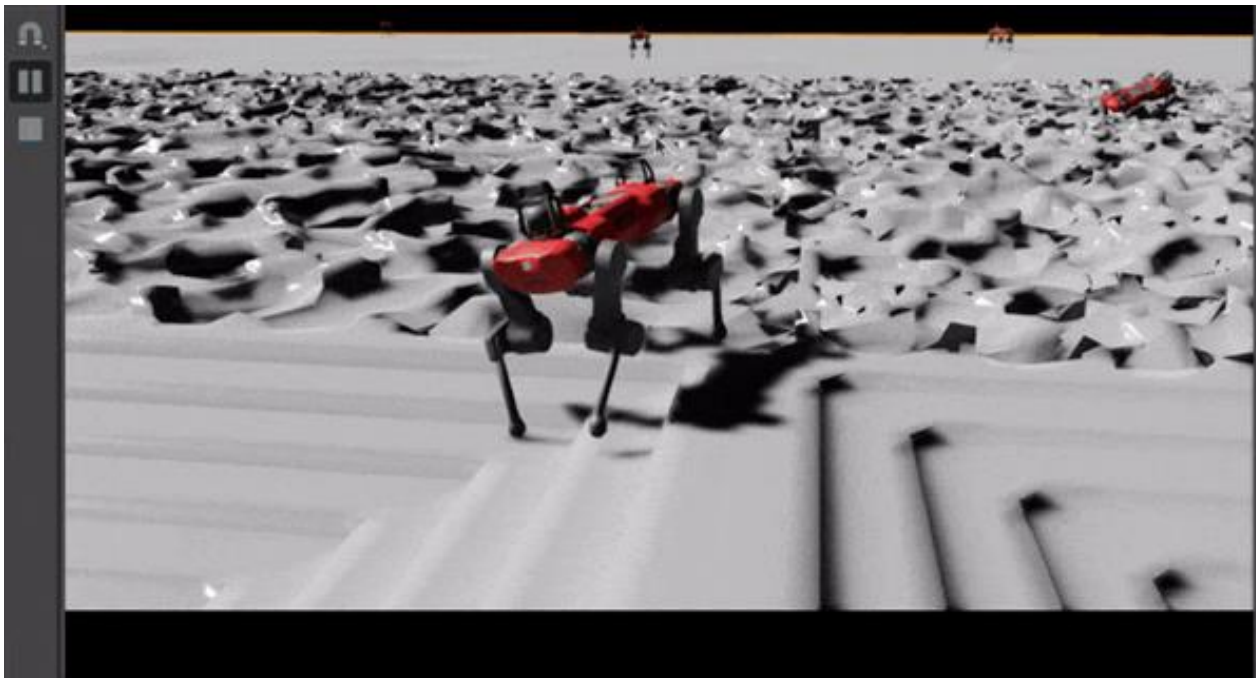


*<https://isaac-sim.github.io/IsaacLab/main/source/overview/environments.html>

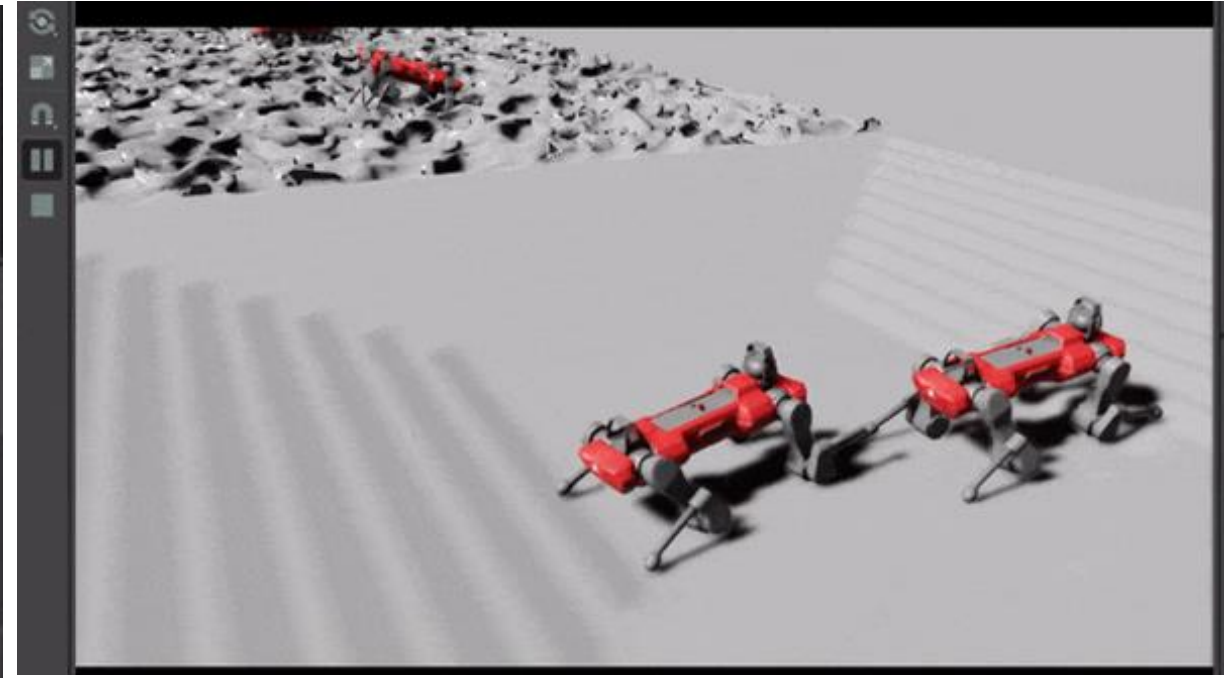
3D Simulator

Nvidia Isaac Simulator

- Isaac Sim and Isaac Lab
 - Comes with baseline policy* you can train and then control a robot to move (just like CARLA)



This policy works



This does not work

*See https://github.com/NVIDIA-Omniverse/OmnisaacGymEnvs/blob/main/docs/examples/rl_examples.md

Writing Software for Robotics

- Robotics requires very complex software
- We deal with the complexity through modularity
- Different from in Simulators, where you can simply control a robot by some API
 - In real-world, usually each part of the robots (camera, arms, other sensors) are connected via some network

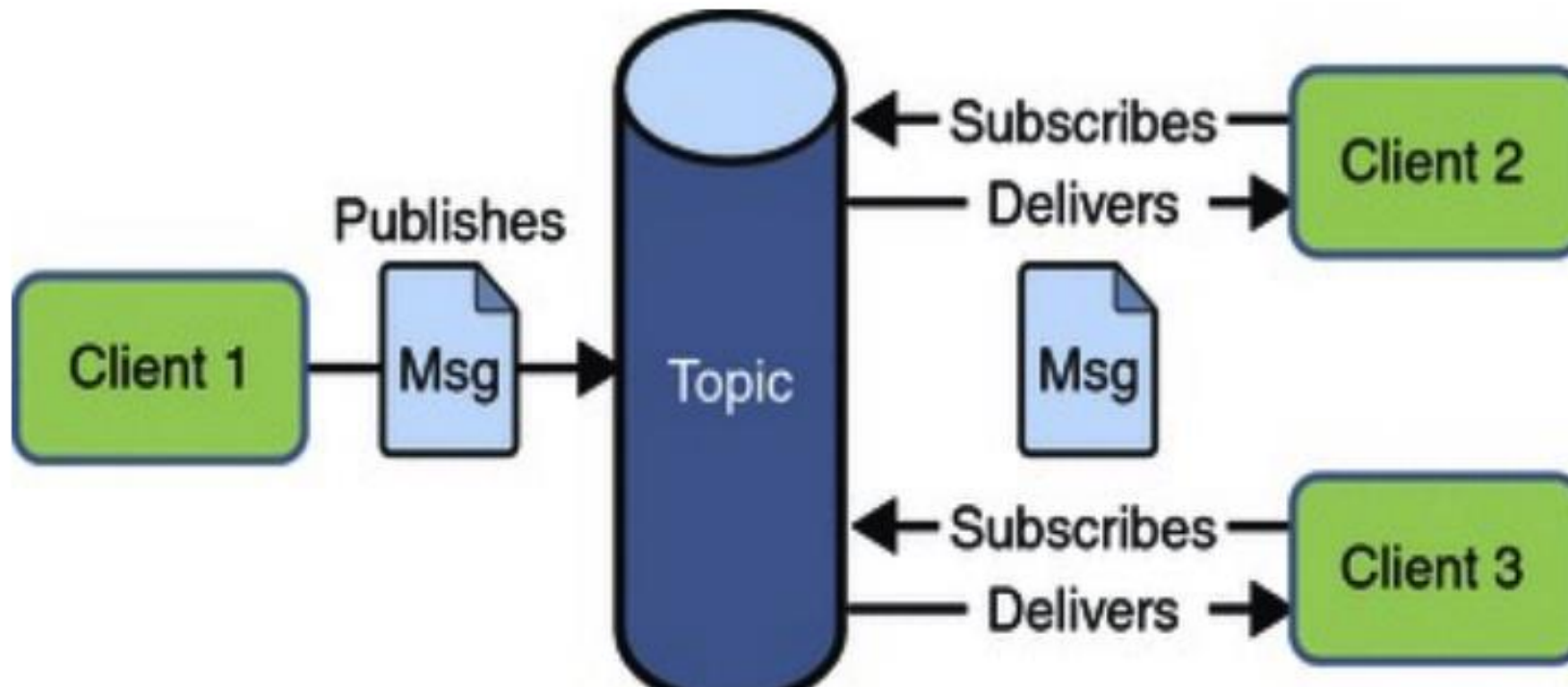
The Publisher/Subscriber Design Pattern

- We divide our software into individual components
- We define “**topics**” (think chat rooms) where components can broadcast information to anyone listening
 - Or a sensor like LiDAR or RGB camera in AD system
- Each component can:
 - Publish: send messages to a topic regardless of whether someone is listening or not
 - Subscribe: receive messages on a topic if anyone is sending them regardless of who

The Publisher/Subscriber Design Pattern

- Example graph for topic -> publish/subscribe

So this means that suppose we have a RGB camera sending out RGB stream to a topic, clients like the robot leg controller and a person detection perception model can access this at the same time



Robot Operating System (ROS)

- An implementation of pub/sub geared towards robotic applications and that is **network-aware**
- Lots of open-source software shared by the community:
 - SLAM (gmapping, amcl)
 - Vision (OpenCV*, PCL, OpenNI)
 - Arm Navigation (MoveIt, MoveIt2)
 - Simulation (Gazebo, CARLA/Isaac Sim also support it)
- Most robots design their APIs in the ROS framework
 - Unitree robots, etc.

* Example: <https://automaticaddison.com/working-with-ros-and-opencv-in-ros-noetic/>

ROS History

- ~2006, Former Google VPs decide to work on Robots
 - Create a company called Willow Garage
 - From this org we get OpenCV, PCL, ROS, PR2 Robot, and many spin out
- 2007-Today
 - Willow awards 11 \$400k PR2 robots to Universities
 - OSRF (Open Source Robotics Foundation) created to maintain ROS and Gazebo
 - 2017 ROS 2 Begins to move ROS out of the lab
 - Address security and robustness concerns
 - ROS is everywhere!



The Main Software Components

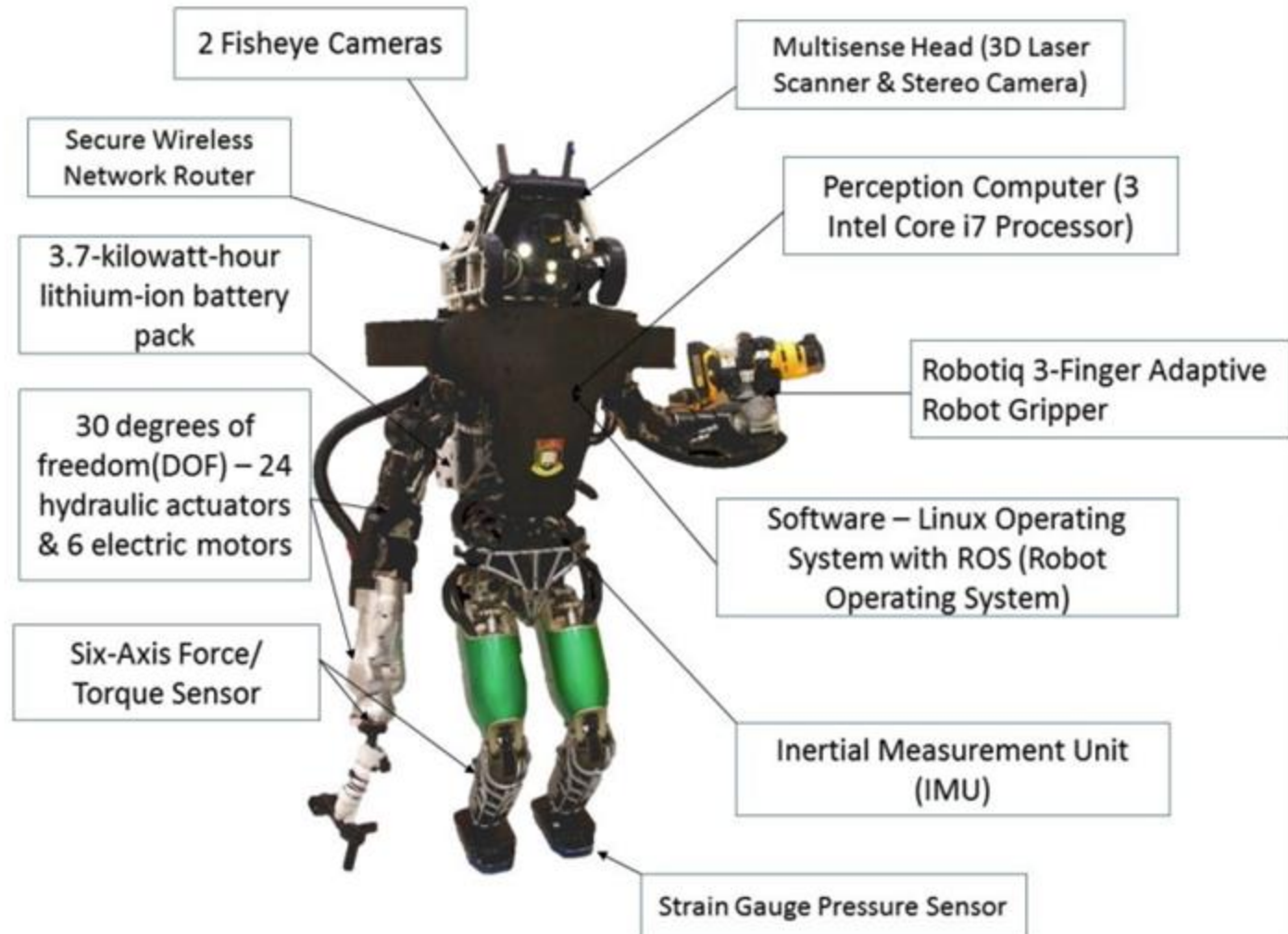
- The core of ROS is the ROS publisher/subscriber bus
 - A **topic** has a message type that is published on the bus. These messages are defined in a yaml file and define the serialization/deserialization format for ROS messages
 - Message types: abstraction away from specific hardware
 - Camera image
 - Laser scan data
 - Motion control
 - ROS has a lot of built in message types. There are lots of pre-defined messages for controlling a robot, distributing sensor data, and understanding the geometry of your robot
 - ROS publishers produce messages slowly or as quickly as they need to
 - A ROS subscriber, subscribes to a topic and then does things with the information

ROS Node

- A process (typically Python or C++) that runs some computation
- The “fundamental” building block
- A ROS **node** can **publish** to one or more **topics**
- That same node can subscribe to other topics
- Many nodes subscribe to topics, process the data, and publish the results
- Run them using `roslaunch <package> <node>`
- Initialize using `rospy.init_node()`
- Some notes
 - Processes / Threads ==> ROS Nodes -- A ROS Node is a self contained execution process, like a program. ROS is really a lot of tooling for running a bunch of programs in parallel

ROS Node

- Sensors and actuators are wrapped in self-contained, reusable software containers called “nodes”

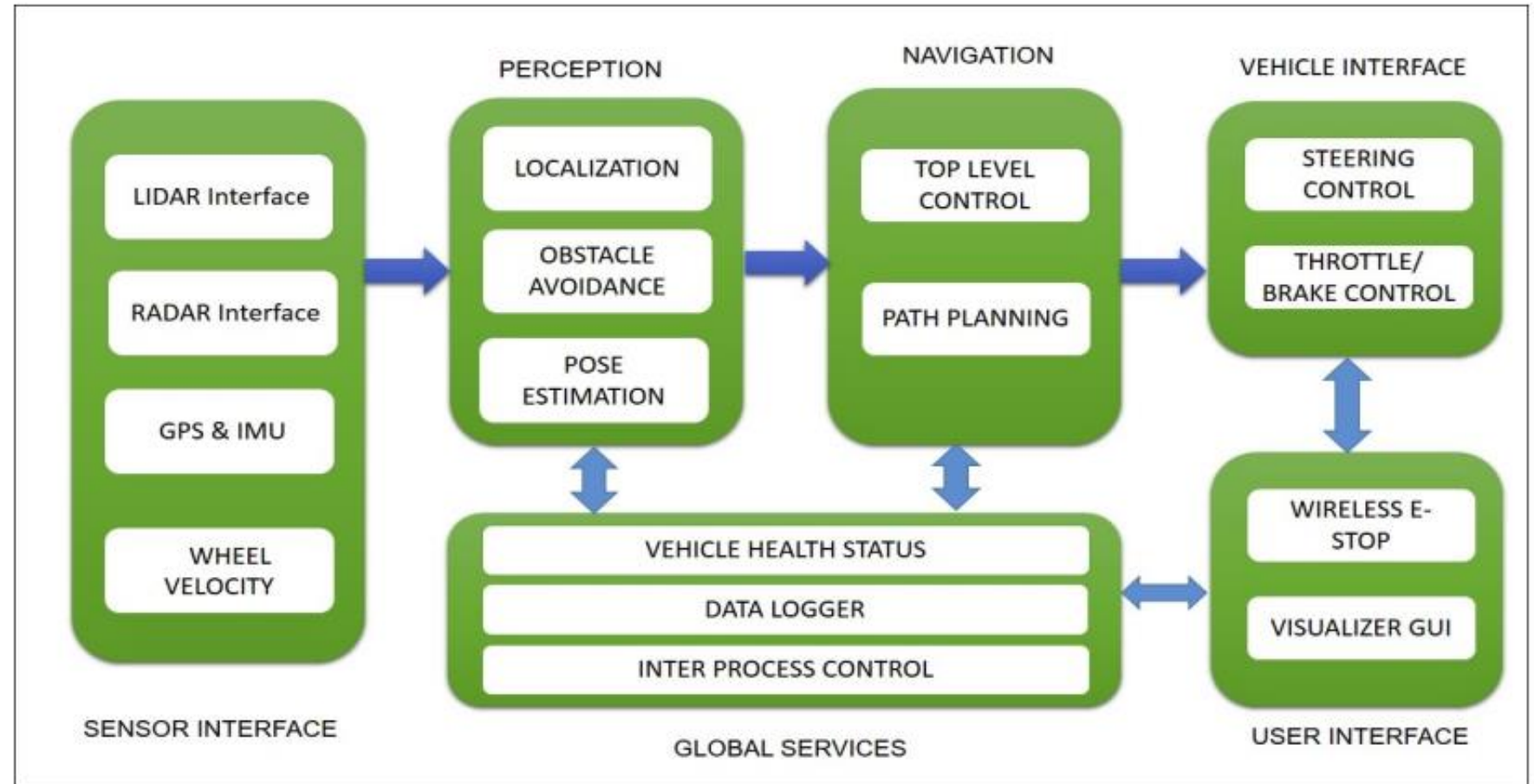


* Robot From DARPA Challenge

ROS

ROS Node

- Higher level operations also become nodes in the ROS computational architecture
 - Perception
 - Navigation



More Concrete Node Examples

- LiDAR node publishes laser scan arrays
- Camera node publishes RGB images (+depth if RGBD) and camera info (resolution, distortion coefficients)
- Mobile robot controller publishes odometry values (e.g. x-y coordinates and velocities, +z for UAVs or underwater vehicles)
- Navigation node **subscribes** to LiDAR and odometry messages, **publishes motion control** messages

ROS Master

- A process that is in charge of coordinating nodes, publishers and subscribers
- Also provides a global parameter server
- Exactly one of them running at any time
- Messages do NOT go through Master (i.e. peer-to-peer)
- Nodes will not be able to find each other without Master
- **All of the above not true in ROS 2 (no more master!)**
 - **Some legacy robots still use ROS 1..**

Sending Messages

```
pub = rospy.Publisher()  
msg = ...  
pub.publish(msg)
```

ROS Messages

- You can check if you are sending messages using the rostopic command line tool:

`rostopic list` – lists all the active topics

`rostopic echo <topic>` – prints messages received on <topic>

`rostopic hz <topic>` – measures topic publishing rate

In ROS 2, the cmd is
\$ `ros2 topic list`

Receiving Messages

```
rospy.Subscriber("chatter", String, callback)  
def callback(msg): ...
```

“chatter” is the topic name

ROS Node - Subscriber

- python code example (rospy)

The callback function is called every time a message is received

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(msg):
    rospy.loginfo("Received: %s", msg.data)

def listener():
    rospy.init_node('listener', anonymous=True)

    rospy.Subscriber("chatter", String, callback)

    rospy.loginfo("Listening on the chatter topic...")

    rospy.spin()

if __name__ == '__main__':
    listener()
```


Using the low-level Communication Tool - DDS

- ROS2 actually uses DDS - So Unitree robots does not use ROS2 but DDS instead
- I like it since it has much smaller overhead compared to ROS2
 - We actually only need Publisher and Subscriber over network - DDS provides them!
 - Here I show you an example code to control the Unitree Dex3 robot hands
 - You need the Subscriber to get the joint states of the hand - is it open? Is it grabbing something?
 - You need the Publisher to send the joint targets to the hand - thumb and index finger touch now!
 - That's it! The rest will be to figure out what joint targets to send at 100Hz, etc.

You don't need
ROS2 to control
Unitree robots.
Use DDS to avoid
overhead

```
# initialize handcmd publisher and handstate subscriber
self.LeftHandState_subscriber = ChannelSubscriber(kTopicDex3LeftState, HandState_)
self.LeftHandState_subscriber.Init()
self.RightHandState_subscriber = ChannelSubscriber(kTopicDex3RightState, HandState_)
self.RightHandState_subscriber.Init()

self.LeftHandCmb_publisher = ChannelPublisher(kTopicDex3LeftCommand, HandCmd_)
self.LeftHandCmb_publisher.Init()
self.RightHandCmb_publisher = ChannelPublisher(kTopicDex3RightCommand, HandCmd_)
self.RightHandCmb_publisher.Init()
```

Next Week – Oct 15, 2025

1. Presentation (Evaluation metrics will be published on Canvas)
2. Project 3 Release
3. For hands-on help with Project 2, please bring your laptop to class next week.



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)